

deephold_db

Ein Handbuch in Lehrbuchform

Vom Datenmodell bis zur Walk-Forward-Replikation

einer quantitativen Forschungsarbeit

deephold_db contributors

June 16, 2026

deephold_db

Ein Handbuch in Lehrbuchform

Vom Datenmodell bis zur Walk-Forward-Replikation
einer quantitativen Forschungsarbeit

Stand: June 16, 2026

Dieses Handbuch ist im Quellcode-Repository unter `docs/handbuch/` abgelegt. Es ist als Lehrmaterial gedacht und setzt Programmierkenntnisse voraus, aber keine Finanzmarktexpertise.

Lizenz des Quellcodes: nicht-kommerziell, kein SLA (siehe `LICENSE` im Repo).

Vorwort

0.1 Wofür dieses Handbuch

Dieses Handbuch dokumentiert das Projekt `deephold_db` – eine Finanzmarktdatenbank für Forschung und privaten Gebrauch. Es ist nicht nur eine technische Referenz, sondern als *Lehrbuch* geschrieben: Es erklärt Konzepte, rechtfertigt Designentscheidungen und führt den Leser Schritt für Schritt durch die Architektur.

Das Handbuch richtet sich an drei Zielgruppen:

- **Entwickler**, die das Projekt verstehen, erweitern oder debuggen wollen. Sie brauchen einen vollständigen Überblick über Architektur, Datenmodell, ETL-Pipeline, Tests und Deployment.
- **Forscher**, die quantitative Finanzanalysen auf Basis der gesammelten Daten durchführen wollen – etwa Walk-Forward-Backtests wie in Kapitel 10 beschrieben.
- **Studierende und Einsteiger**, die ein Gefühl dafür bekommen wollen, wie ein real-existierendes Datenprodukt entsteht – vom Docker-Container bis zur Performance-Metrik.

Merke

Dieses Handbuch setzt Programmierkenntnisse voraus – idealerweise Python und Grundkonzepte relationaler Datenbanken – aber *keine* Finanzmarktexpertise. Alle Fachbegriffe werden bei ihrer ersten Verwendung in einer **Definitionsbox** (blau) erklärt.

0.2 Was `deephold_db` nicht ist

Achtung

Dieses Projekt ist ausdrücklich **kein** Bloomberg- oder Refinitiv-Ersatz. Es deckt bewusst nur Retail- und Research-Bedürfnisse ab:

- **Keine** echten Point-in-Time-Daten – die adjusted-close-Reihen von Yahoo Finance sind *mit aktuellem Wissensstand korrigiert* (look-ahead-Bias bei echten Backtests vor 2010 möglich).
- **Keine** CRSP-äquivalente Aktienverlauf-Datenbank – kein Delisting-Tracking, keine Survivorship-Bias-Korrektur.

- **Keine** Intraday- oder Tick-Daten – alles ist Tagesfrequenz.
- **Keine** echte Continuous-Future-Konstruktion – Front-Month Futures werden einzeln betrachtet, kein Roll-Kalender.
- **Keine** Option-Chains, keine Bond-CUSIPs, keine OTC-Derivate.

Für institutionelle Anwendungen mit Echtzeitdaten, vollständiger PIT-Historie und Compliance-Audit-Trail ist dieses Projekt nicht geeignet. Es ist explizit als *Selbstlern- und Forschungsplattform* konzipiert.

0.3 Wie dieses Handbuch zu lesen ist

Beispiel

Lese-Empfehlungen

- **Entwickler** lesen das Handbuch linear von vorne nach hinten. Die Kapitel bauen aufeinander auf.
- **Forscher** springen direkt zu Kapitel 10 (AEGIS-Replikation) und nutzen Glossar (Anhang A) zum Nachschlagen.
- **Einsteiger** beginnen mit Kapitel 2 (Grundbegriffe) und lesen dann selektiv die Kapitel, die sie interessieren. Das Glossar hilft beim Verständnis der Fachbegriffe.

0.4 Aufbau

Das Handbuch gliedert sich in zehn Hauptkapitel und vier Anhänge:

Kapitel 1 – Einführung Was das Projekt tut, seine Ziele, sein Scope und seine Anwendungsfälle.

Kapitel 2 – Grundlagen Erklärung aller Fachbegriffe, die im restlichen Handbuch vorkommen – von PostgreSQL bis zu CAGR.

Kapitel 3 – Infrastruktur Docker, Docker Compose, das Makefile, Python-Environment und die Bun-Runtime.

Kapitel 4 – Datenmodell Das ER-Diagramm, die 14 Tabellen und ihre Indizes.

Kapitel 5 – Vendor-Adapter Die drei Datenlieferanten FRED, ECB und Yahoo Finance.

Kapitel 6 – ETL-Pipeline Seeder und das query_db.py-Skript.

Kapitel 7 – Notebook-Generator Das Plotly-Notebook und sein Build-Prozess.

Kapitel 8 – OpenTUI-Explorer Das Terminal-UI auf Basis von Bun und React 19.

Kapitel 9 – Tests pytest, bun test und die manuelle Test-Checklist.

Kapitel 10 – Analytics & AEGIS-Replikation Die Walk-Forward-Backtest-Implementierung und ihre Ergebnisse im Vergleich zum Originalpaper.

Anhang A – Glossar Alphabetisches Verzeichnis aller Fachbegriffe.

Anhang B – Befehlsreferenz Alle Makefile-Targets und CLI-Befehle.

Anhang C – Datenbankschema Vollständiges DDL aller Tabellen.

Anhang D – Literatur Bibliographie der zitierten Werke.

0.5 Danksagung

Dieses Projekt wäre ohne die folgenden Open-Source-Bibliotheken und Datenquellen nicht möglich:

- **PostgreSQL 16** – die Datenbank-Engine.
- **SQLAlchemy 2.x** – das ORM.
- **Pandas, Polars, NumPy** – Datenverarbeitung in Python.
- **httpx, tenacity, structlog** – robustes HTTP und Logging.
- **Prefect 2.x** – Workflow-Orchestrierung (vorbereitet).
- **Plotly** – interaktive Plots im Browser.
- **Bun, React 19, OpenTUI** – das TUI-Frontend.
- **FRED, ECB, Yahoo Finance** – die drei Datenlieferanten.
- **AEGIS-Paper** – der Anlass für die Backtest-Replikation.

Vielen Dank an alle Maintainer dieser Projekte.

Contents

Vorwort	iii
0.1 Wofür dieses Handbuch	iii
0.2 Was deephold_db nicht ist	iii
0.3 Wie dieses Handbuch zu lesen ist	iv
0.4 Aufbau	iv
0.5 Danksagung	v
List of Figures	xi
List of Tables	xiii
1 Einführung und Projektüberblick	1
1.1 Das Problem	1
1.2 Die Lösung: deephold_db	1
1.3 Anwendungsfälle	2
1.4 Nicht-Ziele	3
1.5 Architektur auf einen Blick	3
1.6 Was Sie als Nächstes erwartet	3
2 Grundlagen	5
2.1 Datenbank-Grundlagen	5
2.2 Python-Grundlagen	7
2.3 Docker und Container	9
2.4 Networking-Grundlagen	10
2.5 TypeScript- und TUI-Grundlagen	11
2.6 Finanzmarkt-Grundlagen	11
2.7 Was kommt als Nächstes?	15
3 Infrastruktur	17
3.1 Überblick: Was läuft wo?	17
3.2 Docker und Docker Compose	18
3.3 Das Makefile	19
3.4 Python-Environment	21
3.5 Bun und TypeScript	22

3.6	Pfade, Ports, Datenverzeichnisse	22
3.7	Was beim ersten Start passiert	22
3.8	Was als Nächstes?	23
4	Datenmodell	25
4.1	Überblick: Die 14 Tabellen	25
4.2	ER-Diagramm	25
4.3	Stammdaten-Tabellen	26
4.3.1	venues	26
4.3.2	vendors	26
4.3.3	instruments	26
4.3.4	instrument_identifiers	27
4.4	Preis-Tabellen	27
4.4.1	prices_raw – Das unveränderte Original	27
4.4.2	prices_daily – Die harmonisierte Zeitreihe	27
4.5	Makro-Tabellen	28
4.5.1	macro_series und macro_observations	28
4.6	FX- und Bond-Tabellen	28
4.6.1	fx_rates_daily	28
4.6.2	bond_yields	29
4.7	DQ-Tabellen	29
4.7.1	dq_runs und dq_findings	29
4.8	SQLAlchemy-Definitionen	29
4.9	Was als Nächstes?	30
5	Vendor-Adapter	31
5.1	Das Vendor-Interface	31
5.2	FRED – Federal Reserve Economic Data	32
5.3	ECB – European Central Bank SDMX	33
5.4	Yahoo Finance (via yfinance)	35
5.5	Rate-Limiting und Retry	37
5.6	Was als Nächstes?	38
6	ETL-Pipeline	39
6.1	Überblick	39
6.2	Der Seeder-Pattern	39
6.3	Das query_db.py-Skript	40
6.4	Bulk-Seeding mit seed_paper_data.py	41
6.5	DQ-Checks (Vorbereitung)	42
6.6	Idempotenz	42
6.7	Failure-Isolation	43
6.8	Was als Nächstes?	43
7	Notebook-Generator	45

7.1	Überblick: Pipeline	45
7.2	Warum programmatisch generieren?	45
7.3	Der Build-Prozess	45
7.4	Plotly 3×2-Grid	46
7.5	HTML-Export	46
7.6	Was als Nächstes?	47
8	OpenTUI-Explorer	49
8.1	Was ist OpenTUI?	49
8.2	Warum TUI statt GUI?	49
8.3	Architektur des TUI-Layers	50
8.4	Die Komponenten	50
8.4.1	Entry Point: <code>index.tsx</code>	50
8.4.2	Root: <code>App.tsx</code>	51
8.4.3	SeriesList – Linke Spalte	51
8.4.4	SeriesDetail – Rechte Spalte	51
8.5	Die SQL-Queries	52
8.6	Tastatur-Handling	53
8.7	Tests	54
8.8	Was als Nächstes?	54
9	Tests	55
9.1	Test-Philosophie	55
9.2	Python-Tests mit <code>pytest</code>	55
9.2.1	Überblick	55
9.2.2	Test-Fixtures	56
9.2.3	Mocking	56
9.3	TypeScript-Tests mit <code>bun test</code>	57
9.3.1	Überblick	57
9.3.2	Mocking in <code>bun test</code>	57
9.3.3	Integration-Tests	58
9.4	Manuelle Test-Checklist	58
9.5	Code-Qualität	59
9.6	Was als Nächstes?	59
10	Analytics und AEGIS-Paper-Replikation	61
10.1	Das AEGIS-Paper	61
10.1.1	Die 3-Layer-Architektur	61
10.2	Das VAM-Signal	61
10.3	Walk-Forward-Backtest	62
10.4	Performance-Metriken	63
10.5	Das Paper-Universum	64
10.6	Die Ergebnisse	64

10.6.1 Diskussion	64
10.7 Lücken zum vollen Paper	65
10.8 Wie replizieren?	65
10.9 Nächste Schritte zur vollen Replikation	65
10.10 Was als Nächstes?	66
A Glossar	67
B Befehlsreferenz	77
B.1 Makefile-Targets	77
B.2 Python-Skripte	77
B.3 Umgebungsvariablen	78
B.4 Tastatur-Shortcuts (TUI)	78
B.5 pg-Environment (für TUI)	78
B.6 Bun-Befehle	78
C Datenbank-Schema (DDL)	81
C.1 Stammdaten	81
C.2 Preis-Tabellen	82
C.3 Makro-, FX- und Bond-Tabellen	82
C.4 DQ-Tabellen	83
C.5 Indizes im Überblick	84
D Literatur	85

List of Figures

1.1	High-Level-Architektur von <code>deephold_db</code> : Python-Backend füllt die DB über Vendor-Adapter, das TUI und das Notebook lesen direkt aus der DB. Die einzige Kopplung zwischen den Layern ist die Datenbank selbst.	3
3.1	Lokale Infrastruktur: Vier Prozesse, eine Datenbank. Alles läuft auf einer Maschine. PostgreSQL ist der zentrale Hub; Python und Bun kommunizieren nicht direkt, sondern nur über die DB.	17
4.1	Entity-Relationship-Diagramm des <code>deephold_db</code> -Schemas. <code>instruments</code> ist der zentrale Asset-Master; <code>vendors</code> ist die Quelle aller Preise. Makro- und Equity-Daten sind <i>parallel</i> organisiert, nicht in einer Hierarchie.	26
6.1	Der ETL-Flow: Sechs Schritte, jeweils mit klarer Verantwortung. Im aktuellen Projekt sind die Validate-Schritte vorbereitet, aber noch nicht vollständig implementiert (geplant für eine DQ-Iteration).	39
7.1	Build-Pipeline für das Notebook. <code>build_notebook.py</code> erzeugt die <code>.ipynb</code> programmatisch aus Cell-Definitionen. <code>jupyter nbconvert</code> führt die Cells aus und exportiert HTML.	45
7.2	3×2 Plotly-Subplot-Grid im Notebook. Die obere Hälfte zeigt Makro-Daten, die untere Hälfte Equity-Daten.	47
8.1	TUI vs. GUI	49
10.1	Die 3-Layer-Architektur von AEGIS. Im aktuellen Projekt implementieren wir nur Layer 1 (VAM) – Layer 2 (Minimax) und Layer 3 (SLSQP) sind als nächste Iterationen geplant.	62

List of Tables

3.1	Wichtigste Python-Abhängigkeiten	22
3.2	Pfade und Ports aller Komponenten	23
4.1	Alle 14 Tabellen des <code>deephold_db</code> -Schemas	25
5.1	Rate-Limits und Burst pro Vendor. Diese sind in <code>src/deephold_db/vendors/<name>.py</code> konfiguriert.	38
8.1	Tastatur-Bindings des TUI. Die j/k-Variante ist Vim-style für Entwickler, die diese Tasten gewohnt sind.	53
10.1	Performance-Vergleich: VAM-Top10 (unsere Replikation) vs. 5 Benchmarks und das volle AEGIS-Paper. VAM-Top10 schlägt S&P 500 deutlich (CAGR +1,74 %, MaxDD halbiert) und hat niedrigere Volatilität als alle Benchmarks.	64
10.2	Status der Paper-Replikation. Die <i>Daten</i> und <i>Backtest-Infrastruktur</i> sind vollständig; die <i>Strategie</i> selbst ist eine vereinfachte VAM-Lite-Variante.	65
B.1	Alle Makefile-Targets. Aufruf: <code>make <target></code>	77
B.2	Standalone-Skripte. Aufruf jeweils aus dem Repo-Root.	77
B.3	Umgebungsvariablen. In <code>.env</code> setzen – niemals im Quellcode.	78
B.4	Tastatur-Shortcuts im OpenTUI-Explorer.	78
B.5	Postgres-Environment für TUI (<code>tui/src/data/db.ts</code>).	78
B.6	Bun-Befehle im <code>tui/-</code> -Verzeichnis.	79
C.1	Alle Indizes des Schemas. PK = Primärschlüssel, UQ = Unique-Constraint.	84

Chapter 1

Einführung und Projektüberblick

1.1 Das Problem

Quantitative Finanzforschung lebt von Daten – aber an genau diesen Daten scheitern die meisten Hobby- und Kleinprojekte:

- **Datenquellen sind fragmentiert.** Makro-Daten kommen vom FED, Aktien-Daten von Yahoo, Anleihe-Daten vom Treasury, FX-Daten von der EZB, Rohstoff-Daten von der COMEX. Jede Quelle hat ein anderes Format, eine andere API, andere Lizenzbedingungen.
- **Historie ist teuer.** Bloomberg-Terminals kosten 20.000 /Jahr, Refinitiv vergleichbar. Für Hobby-Forscher unbezahlbar.
- **Open Data ist da, aber unhandlich.** FRED, EZB SDMX, Yahoo Finance – alle frei verfügbar, aber jede mit eigener API-Linie, eigenen Rate-Limits, eigenem Datenformat.
- **Reproduzierbarkeit ist schwer.** Wenn der Code zur Datensammlung nicht versioniert und mitprotokolliert ist, weiß man sechs Monate später nicht mehr, welcher Pull welche Daten geliefert hat.
- **Backtesting braucht saubere PIT-Daten.** Eine Aktienkurs-Zeitreihe vom 1. Januar 2020 muss zu jedem späteren Zeitpunkt exakt so aussehen, wie sie tatsächlich an diesem Tag bekannt war – inklusive aller nachträglichen Splits, Dividenden und Restatements.

1.2 Die Lösung: `deephold_db`

`deephold_db` ist eine selbst-gehostete, lokal laufende Finanzmarktdatenbank, die alle genannten Probleme auf eine pragmatische Art adressiert:

Definition

Retail-Research-Datenbank Eine Datenbank, die *für den privaten und Forschungsgebrauch* konzipiert ist. Sie ist nicht für Hochfrequenzhandel, regulatorische Compliance oder institutionelle Vermögensverwaltung gedacht. Sie priorisiert **einfache Bedienung**

und **kostengünstigen Betrieb** über **Latenz** und **vollständige PIT-Treue**.

Die Kernidee: *ein Befehl, eine Datenbank, mehrere Datenquellen, mehrere Frontends*. Konkret bedeutet das:

1. **Ein-Befehl-Setup.** `make init` startet Docker, installiert Python-Dependencies, legt das Datenbankschema an und seedet die ersten Daten. Alles in unter 10 Minuten.
2. **Mehrere Datenquellen, einheitliches Schema.** FRED (FED-Wirtschaftsdaten), EZB SDMX (EUR-Geldmarkt, EUR-Anleihen, EUR-FX) und Yahoo Finance (US-Aktien, ETFs, Indizes) werden über Vendor-Adapter in dasselbe Schema gemappt.
3. **Mehrere Frontends.**
 - **Jupyter-Notebook** (`build_notebook.py`) mit Plotly für interaktive Charts.
 - **OpenTUI-Explorer** (Bun + React 19) für schnelles Durchsuchen der Datenbank im Terminal.
 - **SQL direkt** via `psql` für Ad-hoc-Analysen.
 - **Python-API** (SQLAlchemy ORM) für reproduzierbare Research-Skripte.
4. **Reproduzierbarkeit.** Jede Zeile in der Datenbank trägt einen Zeitstempel (`ingested_at`) und eine `vendor_id`. Damit lässt sich auch Monate später rekonstruieren, woher ein bestimmter Wert stammt.
5. **Walk-Forward-Backtest-Framework.** Eine vereinfachte Version des AEGIS-Algorithmus aus dem quantitativen Finanzpaper von Chakraborty und Singh (2026) demonstriert, wie die gesammelten Daten für systematische Strategieentwicklung genutzt werden können.

1.3 Anwendungsfälle

Beispiel

Typische Anwendungsfälle

- **Portfolio-Analyse:** “Wie hat sich mein 60/40-Aktien/Anleihen-Portfolio in den letzten 20 Jahren entwickelt?”
- **Makro-Research:** “Wie korreliert die US-Arbeitslosenquote mit S&P-500-Renditen in den letzten 10 Jahren?”
- **Faktor-Analyse:** “Lohnt sich Value-Investing in Small-Caps noch?”
- **Backtest-Vorbereitung:** “Lade mir 20 Jahre S&P 500- und NASDAQ-Daten, damit ich meinen Mean-Reversion-Filter darauf testen kann.”
- **Bildungsprojekte:** “Ich will meinen Studierenden zeigen, wie ein professionelles Datenprodukt aussieht.”

1.4 Nicht-Ziele

Achtung

Folgende Funktionen sind bewusst nicht Teil dieses Projekts:

- Echtzeitdaten oder Intraday-Ticks.
- Echtes Point-in-Time mit allen Splits/Dividenden zum jeweiligen historischen Zeitpunkt (nur adjusted-close, keine nachträglichen Restatements).
- Bond-CUSIP-Level-Daten (nur aggregierte Anleihe-ETFs oder Makro-Yields).
- Multi-Vendor-Reconciliation (Preisvergleiche zwischen Datenquellen).
- Historische Index-Mitgliedschaften (“Wann wurde AAPL in den S&P 500 aufgenommen?”).
- Historische Credit-Ratings.

1.5 Architektur auf einen Blick

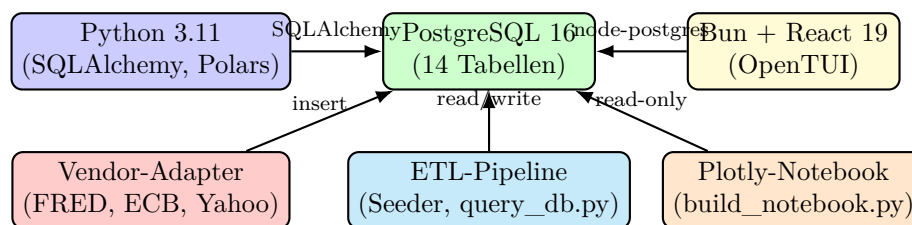


Figure 1.1: High-Level-Architektur von deephold_db: Python-Backend füllt die DB über Vendor-Adapter, das TUI und das Notebook lesen direkt aus der DB. Die einzige Kopplung zwischen den Layern ist die Datenbank selbst.

Abbildung 1.1 zeigt die drei Schichten des Projekts:

1. **Python-Backend** (links): Enthält die Vendor-Adapter (FRED, ECB, Yahoo), die ETL-Pipeline (Seeder, query_db.py) und die Analytics-Module (Returns, Metriken, VAM-Backtest). Spricht via SQLAlchemy mit der DB.
2. **PostgreSQL-Datenbank** (Mitte): Single Source of Truth. 14 Tabellen – venues, vendors, instruments, instrument_identifiers, trading_calendars, prices_raw, prices_daily, corporate_actions, fx_rates_daily, bond_yields, macro_series, macro_observations, dq_runs, dq_findings.
3. **TypeScript-TUI** (rechts): OpenTUI-Explorer in Bun. Liest die DB via node-postgres, kein Python im Hot-Path. Maximale Performance für den Endbenutzer.

1.6 Was Sie als Nächstes erwartet

Das nächste Kapitel –2– bringt Sie auf den nötigen Wissensstand, falls Sie mit einem der verwendeten Konzepte noch nicht vertraut sind. Falls Sie bereits Erfahrung mit PostgreSQL, Docker, Python und TypeScript haben, können Sie direkt mit Kapitel 3 fortfahren.

Chapter 2

Grundlagen

Dieses Kapitel erklärt alle Fachbegriffe, die in den Folgekapiteln verwendet werden. Wer mit allen Konzepten vertraut ist, kann es überspringen und bei Unklarheiten als Nachschlagewerk nutzen.

2.1 Datenbank-Grundlagen

Definition

Relationale Datenbank Eine *relationale Datenbank* organisiert Daten in *Tabellen* (auch *Relationen* genannt) mit Zeilen (Datensätze) und Spalten (Attribute). Zwischen den Tabellen bestehen *Beziehungen* über *Fremdschlüssel* (*foreign key*). Die meisten modernen Datenbanken – einschließlich PostgreSQL – sprechen die standardisierte Abfragesprache SQL.

Definition

SQL (Structured Query Language) SQL ist die standardisierte Abfragesprache für relationale Datenbanken. SQL unterteilt sich in Untergruppen:

- **DDL** (Data Definition Language): CREATE TABLE, ALTER TABLE, DROP TABLE.
- **DML** (Data Manipulation Language): SELECT, INSERT, UPDATE, DELETE.
- **DCL** (Data Control Language): GRANT, REVOKE.

Definition

PostgreSQL PostgreSQL – oft kurz *Postgres* – ist ein *objekt-relationales* Datenbankmanagementsystem (ORDBMS). Es ist quelloffen, kostenlos, seit über 30 Jahren in Entwicklung und gilt als eine der stabilsten und funktionsreichsten Open-Source-Datenbanken. PostgreSQL unterstützt JSON/JSONB, Volltextsuche, Window Functions und vieles mehr.

Beispiel

PostgreSQL-Verbindung in Python

```

1  import psycopg
2
3  conn = psycopg.connect(
4      host="localhost",
5      port=5432,
6      user="deephold",
7      password="deephold",
8      dbname="deephold",
9  )
10 with conn.cursor() as cur:
11     cur.execute("SELECT COUNT(*) FROM macro_observations")
12     print(cur.fetchone())

```

Definition

Index Ein *Index* ist eine zusätzliche Datenstruktur, die das schnelle Suchen in einer Tabelle ermöglicht – analog zum Stichwortverzeichnis eines Buchs. Ein Index auf Spalte *X* beschleunigt Abfragen mit `WHERE X = ...`, verlangsamt aber `INSERT`-Operationen und kostet zusätzlichen Speicherplatz. Daher werden nur Spalten indiziert, die häufig in `WHERE`-Klauseln vorkommen.

Beispiel

Index-Definition

```

1  CREATE INDEX ix_prices_daily_date
2      ON prices_daily (date);
3
4  CREATE UNIQUE INDEX ix_instrument_identifiers_scheme_value
5      ON instrument_identifiers (scheme, value);

```

Definition

JSONB JSONB ist die binäre Variante des JSON-Datentyps in PostgreSQL. Werte werden nicht als Text, sondern als effizient gepackte interne Struktur gespeichert. Das ermöglicht Indexierung auf JSON-Feldern und schnelle Pfad-Queries (`data->'field'`).

2.2 Python-Grundlagen

Definition

Python Python ist eine interpretierte, dynamisch typisierte Hochsprachen-Programmiersprache. Sie wird im Data-Science-Umfeld besonders wegen ihrer einfachen Syntax, ihrer umfangreichen Standardbibliothek und ihres dichten Ökosystems an Drittbibliotheken geschätzt. Das Projekt verwendet Python in Version 3.11 oder neuer.

Definition

Poetry Poetry ist ein modernes Tool für *Dependency-Management* in Python-Projekten. Es verwaltet die `pyproject.toml`-Datei, legt virtuelle Environments automatisch an und löst transitive Abhängigkeiten auf. In diesem Projekt wird es durch `pip` und ein `venv`-Verzeichnis ersetzt (siehe Kapitel 3), weil Poetry nicht überall verfügbar ist.

Definition

Pandas Pandas ist die Standard-Bibliothek für tabellarische Datenverarbeitung in Python. Die zentrale Datenstruktur ist das *DataFrame* – eine beschriftete, zweidimensionale Tabelle. Pandas wird für Daten- Cleaning, Resampling, Grouping und vieles mehr verwendet.

Definition

Polars Polars ist eine neuere, schnellere Alternative zu Pandas, implementiert in Rust. Polars nutzt *Lazy Evaluation* und *Columnar Storage*, was es auf großen Datensätzen drastisch schneller macht. Die API ähnelt Pandas, ist aber stärker typisiert.

Beispiel

Polars vs. Pandas – ein einfaches Beispiel

```
1 import polars as pl
2 import pandas as pd
3
4 # Beide erzeugen das gleiche Ergebnis:
5 df_pl = pl.DataFrame({"a": [1, 2, 3], "b": [4.0, 5.0, 6.0]})
6 df_pd = pd.DataFrame({"a": [1, 2, 3], "b": [4.0, 5.0, 6.0]})
7
8 # Polars:
9 mean_b = df_pl["b"].mean() # Lazy, gibt eine Expression zur ck
10 print(df_pl.select(pl.col("b").mean())) # 5.0
11
12 # Pandas:
13 mean_b = df_pd["b"].mean() # Eager, gibt einen float zur ck
14 print(mean_b) # 5.0
```

Definition

SQLAlchemy SQLAlchemy ist ein *ORM-Toolkit* (Object-Relational Mapper) für Python. Es erlaubt, Datenbank-Tabellen als Python-Klassen zu modellieren und Abfragen in Python-Syntax zu schreiben, ohne rohes SQL zu schreiben. SQLAlchemy unterstützt zwei Stile:

- **Core:** niedrigschwellige SQL-Bausteine (`select()`, `insert()` etc.).
- **ORM:** hochschwellige, objekt-orientierte Zugriffsschicht mit Klassen für jede Tabelle.

In diesem Projekt wird die ORM-Schicht verwendet.

Beispiel**SQLAlchemy-Definition einer Tabelle**

```

1  from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
2
3  class Base(DeclarativeBase):
4      pass
5
6  class Instrument(Base):
7      __tablename__ = "instruments"
8      instrument_id: Mapped[int] = mapped_column(primary_key=True)
9      asset_class: Mapped[str] = mapped_column()
10     name: Mapped[str] = mapped_column()
11     # ...

```

Definition

Alembic Alembic ist das offizielle Migrationswerkzeug für SQLAlchemy. Es verwaltet *Schema-Migrationen* – das Hinzufügen, Ändern oder Löschen von Tabellen und Spalten, ohne Daten zu verlieren. Jede Migration ist eine Python-Datei mit zwei Funktionen: `upgrade()` und `downgrade()`.

Beispiel**Alembic-Migration – sehr vereinfacht**

```

1  def upgrade() -> None:
2      op.create_table(
3          "macro_series",
4          sa.Column("series_id", sa.Text, primary_key=True),
5          sa.Column("name", sa.Text),
6          # ...
7      )
8
9  def downgrade() -> None:
10     op.drop_table("macro_series")

```

2.3 Docker und Container

Definition

Container Ein *Container* ist eine isolierte Laufzeit-Umgebung für eine Anwendung. Anders als bei virtuellen Maschinen wird nicht der gesamte Rechner virtualisiert, sondern nur die *Anwendungs-Layer* (Binaries, Libraries, Konfiguration). Container starten in Sekunden, nicht in Minuten, und sind sehr ressourcenschonend.

Definition

Docker Docker ist die populärste Container-Engine. Sie packt eine Anwendung in ein *Image* (eine Art Bauplan) und startet daraus *Container*. Ein Image wird durch ein Dockerfile beschrieben, ein Container ist eine laufende Instanz eines Images.

Definition

Docker Compose Docker Compose ist ein Werkzeug zur Definition und zum Start *mehrerer* Container als zusammengehörige Anwendung. Die Konfiguration erfolgt in einer `docker-compose.yml`-Datei. Mit `docker compose up -d` startet man alle Services im Hintergrund; mit `docker compose down` stoppt man sie wieder.

Beispiel

docker-compose.yml – Auszug

```

1  services:
2    postgres:
3      image: postgres:16-alpine
4      environment:
5        POSTGRES_USER: deephold
6        POSTGRES_PASSWORD: deephold
7        POSTGRES_DB: deephold
8      ports:
9        - "5432:5432"
10     healthcheck:
11       test: ["CMD-SHELL", "pg_isready -U deephold"]
12       interval: 5s

```

Definition

Docker Volume Ein *Volume* ist ein persistenter Speicherbereich, der vom Container unabhängig existiert. Wird ein Container gelöscht, bleiben Daten im Volume erhalten. `deephold_db` definiert ein Volume `postgres_data` für die Datenbank, damit die Daten Container-Updates überleben.

2.4 Networking-Grundlagen

Definition

HTTP (Hypertext Transfer Protocol) HTTP ist das Standard-Protokoll für die Kommunikation zwischen Web-Clients und Web-Servern. Datenquellen wie FRED und Yahoo Finance bieten ihre Daten über *RESTful HTTP*-APIs an. Ein typischer Aufruf ist `GET https://api.example.com/data?param=value`, der mit einem JSON-Array oder XML-Dokument beantwortet wird.

Definition

Rate Limiting Die meisten öffentlichen APIs begrenzen, wie viele Anfragen ein einzelner Client pro Zeiteinheit stellen darf – das sogenannte *Rate Limiting*. Wird das Limit überschritten, antwortet der Server mit HTTP 429 (Too Many Requests). Das Projekt implementiert ein *Token-Bucket*-Verfahren, bei dem jeder Vendor ein eigenes Bucket mit konfigurierbarer Rate und Burst-Größe hat.

Beispiel

Token-Bucket in Python

```
1 import time
2
3 class TokenBucket:
4     def __init__(self, rate_per_min: int, burst: int):
5         self.rate = rate_per_min / 60.0    # tokens per second
6         self.burst = burst
7         self.tokens = float(burst)
8         self.last = time.monotonic()
9
10    def take(self, n: int = 1) -> None:
11        now = time.monotonic()
12        elapsed = now - self.last
13        self.last = now
14        self.tokens = min(self.burst, self.tokens + elapsed * self.
15                           rate)
16        if self.tokens < n:
17            # Blockierend warten
18            time.sleep((n - self.tokens) / self.rate)
19            self.tokens = 0
20        else:
21            self.tokens -= n
```


2.5 TypeScript- und TUI-Grundlagen

Definition

TypeScript TypeScript ist eine Obermenge von JavaScript, die statische Typen hinzufügt. TypeScript-Code wird zu JavaScript compiliert und läuft dann in jeder JavaScript-Engine (z. B. Node.js oder Bun).

Definition

Bun Bun ist eine moderne JavaScript-/TypeScript-Runtime, ähnlich wie Node.js, aber deutlich schneller beim Start und bei der Installation von Dependencies. Bun enthält einen eingebauten *Paketmanager*, einen *Test-Runner* und einen *Bundler*. Bun ist die Standard-Runtime für das in diesem Projekt verwendete OpenTUI-Framework.

Definition

Terminal User Interface (TUI) Eine *TUI* ist ein textbasiertes Interface, das im Terminal läuft – im Gegensatz zu einem GUI (grafisches User Interface). Im Gegensatz zu einem Kommandozeilen-Tool, das nur Text ein- und ausgibt, bietet eine TUI ein *interaktives* Layout mit *Widgets* wie Listen, Buttons, Formularfeldern und Charts. Beispiele: vim, htop, lazygit.

Definition

OpenTUI OpenTUI ist ein von Anomaly (<https://anoma.ly>) entwickeltes TUI-Framework. Es besteht aus einem in Zig geschriebenen *Core* und TypeScript-Bindings mit React- und Solid.js-Support. Der Kern nutzt Yoga (Facebooks Flexbox-Engine), um Layouts zu berechnen – genau wie im Web – und rendert das Ergebnis in einen Terminal-Buffer.

Definition

Yoga (Flexbox-Layout) Yoga ist Facebooks plattformübergreifender Layout-Algorithmus, der *flexbox* – das CSS-Layout-System – in native Bibliotheken portiert. OpenTUI nutzt Yoga, um komplexe Layouts (Sidebars, Header, Footer, mehrspaltige Inhalte) pixelgenau im Terminal zu positionieren.

2.6 Finanzmarkt-Grundlagen

Definition

Adjusted Close Der *adjusted Close* ist der Schlusskurs eines Wertpapiers, *nachbereinigt* um alle Kapitalmaßnahmen, die zwischen Kaufzeitpunkt und aktuellem Zeitpunkt stattgefunden haben – vor allem Splits und Dividenden. Wenn eine Aktie am Tag X zu 100 schließt und am Folgetag einen 2:1-Split macht, dann steht der adjusted Close für Tag X nicht bei 100, sondern bei 50. Backtests, die adjusted close verwenden, sind – in der

Retail-Praxis – korrekt für historische Investoren.

Definition

Log-Return vs. Simple Return Es gibt zwei gängige Arten, Renditen zu berechnen:

- **Simple Return:** $R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$
- **Log-Return:** $r_t = \ln \frac{P_t}{P_{t-1}}$

Log>Returns sind *zeitadditiv* – der Log-Return über zwei Tage ist die Summe der täglichen Log>Returns. Dies vereinfacht viele Berechnungen. Bei kleinen Returns ($|r_t| < 5\%$) sind die beide Werte praktisch identisch.

Definition

Volatilität (Vol) Die *Volatilität* ist die annualisierte Standardabweichung der Renditen. Sie ist das Standardmaß für das Risiko eines Wertpapiers oder Portfolios. Übliche Symbole: σ (Sigma) oder vol.

Beispiel

Volatilität berechnen

```
1 import math
2 import statistics
3
4 # Tägliche Log>Returns einer Aktie
5 daily_returns = [0.01, -0.02, 0.005, 0.015, -0.01, 0.0, 0.02]
6
7 # Standardabweichung der täglichen Returns
8 daily_vol = statistics.stdev(daily_returns) # 0.0124
9
10 # Annualisiert (252 Handelstage)
11 annual_vol = daily_vol * math.sqrt(252) # 0.196 = 19.6%
```

Definition

CAGR (Compound Annual Growth Rate) Die *CAGR* ist die durchschnittliche jährliche Wachstumsrate eines Investments über einen bestimmten Zeitraum, unter Annahme eines geometrischen (zinseszinsartigen) Wachstums:

$$\text{CAGR} = \left(\frac{V_{\text{end}}}{V_{\text{start}}} \right)^{1/n} - 1$$

wobei n die Anzahl der Jahre ist. Ein Investment, das von 100 auf 200 in 5 Jahren wächst, hat eine CAGR von $\sqrt[5]{200/100} - 1 \approx 14,87\%$.

Definition

Sharpe Ratio Die *Sharpe Ratio* misst die *renditebereinigte Performance* eines Investments – die Überrendite pro Einheit Gesamtrisiko:

$$\text{Sharpe} = \frac{\bar{R}_p - R_f}{\sigma_p}$$

wobei $\bar{R}_p$ die annualisierte Portfoliorendite, R_f der risikofreie Zins (oft 4 % p. a.) und σ_p die annualisierte Volatilität ist. Höher ist besser; eine Sharpe > 1 gilt als gut, Sharpe > 2 als exzellent.

Definition

Sortino Ratio Eine Variante der Sharpe Ratio, die nur die *Abwärtsvolatilität* (negative Renditen) als Risikomaß verwendet. Sie ist aussagekräftiger für asymmetrische Strategien, die gerne hohe positive Volatilität (große Gewinne) in Kauf nehmen, aber negative Volatilität (Verluste) minimieren wollen.

Definition

Maximum Drawdown (MaxDD) Der *Maximum Drawdown* ist der größte prozentuale Verlust vom einem vorherigen Höchststand bis zu einem nachfolgenden Tiefststand:

$$\text{MaxDD} = \min_t \frac{V_t - V_{\text{peak}}}{V_{\text{peak}}}$$

Ein MaxDD von –30 % bedeutet, dass das Portfolio irgendwann 30 % unter seinem vorherigen Höchststand lag. Dies ist das wichtigste *Tail-Risk*-Maß.

Definition

Walk-Forward-Backtest Ein *Walk-Forward-Backtest* ist die realistischste Form der Backtest-Validierung. Anders als bei einem klassischen *In-Sample*-Backtest wird das Modell nicht auf denselben Daten trainiert und evaluiert, auf denen es entwickelt wurde. Stattdessen:

1. Trainiere auf $[t - L, t]$ (z. B. 12 Monate)
2. Evaluiere auf $[t, t + 1]$ (z. B. 1 Monat)
3. Wandere ein Fenster weiter, wiederhole.

So bekommt das Modell zu jedem Zeitpunkt nur Daten, die es auch *tatsächlich* gehabt hätte – kein *Look-Ahead-Bias*.

Definition

Look-Ahead-Bias *Look-Ahead-Bias* entsteht, wenn ein Backtest Informationen verwendet, die zum Entscheidungszeitpunkt in der Realität noch nicht verfügbar gewesen wären – zum Beispiel ein Split, der erst nach dem Backtest-Datum in den Preisdaten nachträglich eingerechnet wurde. Walk-Forward-Tests eliminieren Look-Ahead-Bias vollständig.

Definition

Survivorship-Bias *Survivorship-Bias* ist die Tendenz, nur die *überlebenden* Unternehmen in einem Index zu betrachten. Wenn Sie z. B. die durchschnittliche Rendite der heutigen S&P-500-Mitglieder in der Vergangenheit berechnen, ignorieren Sie all die Unternehmen, die zwischenzeitlich pleitegingen oder delistet wurden. Das überschätzt die wahre historische Performance.

Definition

Momentum *Momentum* ist die Tendenz von Aktien, die in der Vergangenheit gut gelaufen sind, dies auch in der nahen Zukunft zu tun – und umgekehrt. Die klassische Momentum-Strategie (Jegadeesh & Titman, 1993) kauft die Top-10 % der zurückliegenden 12 Monate und leert die unteren 10 %. Momentum ist eines der am besten dokumentierten Anomalien in der Finanzmarktforschung.

Definition

VAM (Volatility-Adjusted Momentum) Die *VAM* (manchmal auch *Sharpe-Momentum* genannt) normalisiert die historische Rendite durch die historische Volatilität:

$$\text{VAM}_i = \frac{R_i}{\sigma_i}$$

Eine Aktie mit 50 % Jahresrendite und 30 % Volatilität hat einen VAM von 1,67. Eine andere mit 50 % Rendite aber 60 % Volatilität hat einen VAM von 0,83 – sie ist bei gleicher Rendite deutlich riskanter. VAM hilft, den *Winner's Curse* zu vermeiden: Aktien, die nur wegen hoher Beta gut laufen, werden aussortiert.

Definition

Skip-Month Beim Walk-Forward-Backtest ist es üblich, den letzten Monat vor dem Re-Balancing-Datum *auszulassen* (das sogenannte *Skip-Month*). Grund: Die jüngsten 21 Tage enthalten oft Mean-Reversion-Effekte und Mikrostruktur-Rauschen, die die Signale verzerren würden. Das AEGIS-Paper und dieses Projekt verwenden ein 21-Tage-Skip (Default).

Definition

Friction / Transaction Costs *Friction* (auch *Transaction Costs*) sind die *Transaktionskosten* eines Backtests – typischerweise als Bps (Basispunkte, 1 Bp = 0,01 %) pro umgesetztem Dollar ausgedrückt. Das AEGIS-Paper verwendet 10 Bp pro Handel, was etwa 1 Bp pro Monat-Portfolio-Turnover entspricht.

2.7 Was kommt als Nächstes?

Mit diesen Grundlagen bewaffnet, können Sie sich auf die Infrastruktur (Kapitel [3](#)) stürzen. Dort sehen wir, wie alle diese Komponenten in einem lauffähigen System zusammenspielen.

Chapter 3

Infrastruktur

Dieses Kapitel zeigt, wie alle Komponenten – Datenbank, Python, TypeScript, Tools – auf einer Maschine zusammenstarten und miteinander kommunizieren. Es ist die Grundlage, auf der alle folgenden Kapitel aufbauen.

3.1 Überblick: Was läuft wo?

Abbildung 3.1 zeigt, welche Prozesse auf der Entwicklungsmaschine laufen und wie sie miteinander sprechen.

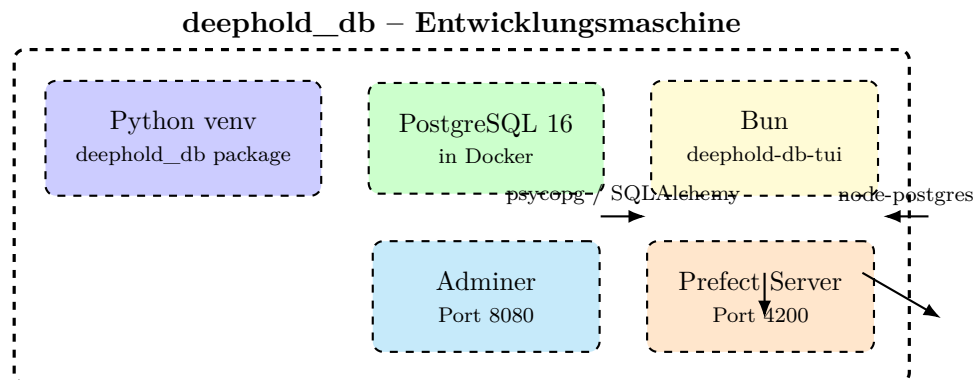


Figure 3.1: Lokale Infrastruktur: Vier Prozesse, eine Datenbank. Alles läuft auf einer Maschine. PostgreSQL ist der zentrale Hub; Python und Bun kommunizieren nicht direkt, sondern nur über die DB.

Definition

localhost / 127.0.0.1 *Localhost* ist der Rechnername, der immer auf den *aktuellen Rechner* selbst verweist. Die IP-Adresse 127.0.0.1 ist die *Loopback*-Adresse. Wenn ein Prozess auf Localhost hört, ist er nur für Prozesse auf derselben Maschine erreichbar – nicht für andere Computer im Netz. Dies ist die sicherste *Standardtopologie* für Entwicklung.

3.2 Docker und Docker Compose

Definition

docker-compose.yml Die Datei `docker-compose.yml` ist die *Single Source of Truth* für alle containerisierten Services. Sie listet jeden Service mit Image, Umgebungsvariablen, Ports, Volumes und Healthcheck auf.

```

1  name: deephold-db
2
3  services:
4    postgres:
5      image: postgres:16-alpine
6      container_name: deephold_pg
7      restart: unless-stopped
8      environment:
9        POSTGRES_USER: ${POSTGRES_USER:-deephold}
10       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-deephold}
11       POSTGRES_DB: ${POSTGRES_DB:-deephold}
12     ports:
13       - "${POSTGRES_PORT:-5432}:5432"
14     volumes:
15       - postgres_data:/var/lib/postgresql/data
16     healthcheck:
17       test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-deephold} -d $
18         {POSTGRES_DB:-deephold}"]
19       interval: 5s
20       timeout: 5s
21       retries: 10
22
23     adminer:
24       image: adminer:4
25       container_name: deephold_adminer
26       restart: unless-stopped
27       ports:
28         - "${ADMINER_PORT:-8080}:8080"
29       environment:
30         ADMINER_DEFAULT_SERVER: postgres
31       depends_on:
32         postgres:
33           condition: service_healthy
34
35     prefect-server:
36       image: prefecthq/prefect:2.16-python3.11
37       container_name: deephold_prefect
38       restart: unless-stopped
39       command: prefect server start --host 0.0.0.0
40       ports:
41         - "${PREFECT_PORT:-4200}:4200"

```



```

41     environment:
42         PREFECT_HOME: /opt/prefect
43         PREFECT_SERVER_API_URL: http://localhost:4200/api
44     volumes:
45         - prefect_data:/opt/prefect
46
47 volumes:
48     postgres_data:
49     prefect_data:

```

Listing 3.1: Vollständige docker-compose.yml

Listing 3.1 zeigt die vollständige docker-compose.yml. Bemerkenswert:

- **Drei Services:** postgres (Datenbank), adminer (Web-UI für SQL), prefect-server (Workflow-Orchestrierung).
- **Zwei Volumes:** postgres_data und prefect_data für persistente Daten.
- **Healthcheck** auf Postgres – der adminer-Service startet erst, wenn Postgres bereit ist (condition: service_healthy).
- **Umgebungsvariablen** mit Defaults: \${POSTGRES_USER:-deephold} – falls die Variable nicht gesetzt ist, wird der Default deephold verwendet.
- **Image-Tags:** postgres:16-alpine – die -alpine-Variante nutzt eine minimale Linux-Distribution (Alpine) und ist entsprechend klein.

3.3 Das Makefile

Das Makefile ist die *einzigste CLI*, die man kennen muss. Es abstrahiert über alle Details – Docker, Python, Bun, Alembic, Tests.

```

1  .PHONY: help
2  help:  ## Zeige alle Targets
3          @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) | \
4          awk 'BEGIN {FS = ":.*?## "}; {printf "\033[36m%-20s\033[0m %s\n", $$1, $$2}'
5
6  .PHONY: init
7  init:  ## Komplett-Setup: deps + Stack + Migrations
8          poetry install
9          $(COMPOSE) up -d
10         @echo "Warte auf Postgres ..."
11         @for i in $(seq 1 30); do \
12             $(COMPOSE) exec -T postgres pg_isready -U ${POSTGRES_USER:-
13                 deephold} >/dev/null 2>&1 && break; \
14             sleep 1; \
15         done
16         $(MAKE) migrate

```

```
16         @echo "Init fertig. UI: Adminer :8080, Prefect :4200"
17
18 .PHONY: up
19 up: ## docker-compose Stack starten
20     $(COMPOSE) up -d
21
22 .PHONY: down
23 down: ## docker-compose Stack stoppen
24     $(COMPOSE) down
25
26 .PHONY: migrate
27 migrate: ## Alembic-Migrationen anwenden
28     DATABASE_URL=$(DB_DSN) $(PYTHON) -m alembic upgrade head
29
30 .PHONY: revision
31 revision: ## Neue Alembic-Revision (Usage: make revision m="...")
32     DATABASE_URL=$(DB_DSN) $(PYTHON) -m alembic revision --
33         autogenerate -m "$(m)"
34
35 .PHONY: test
36 test: ## pytest + pandera schemas
37     DATABASE_URL=$(DB_DSN) $(PYTHON) -m pytest
38
39 .PHONY: tui
40 tui: ## OpenTUI-Explorer starten (Bun erforderlich)
41     @if ! command -v bun >/dev/null 2>&1; then \
42         echo "Bun nicht installiert. Install: curl -fsSL https
43             ://bun.sh/install | bash"; \
44         exit 1; \
45     fi
46     cd tui && bun install --silent && bun run start
47
48 .PHONY: tui-test
49 tui-test: ## TUI-Tests (Unit + Integration)
50     @if ! command -v bun >/dev/null 2>&1; then \
51         echo "Bun nicht installiert. Install: curl -fsSL https
52             ://bun.sh/install | bash"; \
53         exit 1; \
54     fi
55     cd tui && bun test
56
57 .PHONY: format
58 format: ## ruff format
59     $(PYTHON) -m ruff format .
60
61 .PHONY: lint
62 lint: ## ruff check
63     $(PYTHON) -m ruff check .
```

```

61
62 .PHONY: typecheck
63 typecheck: ## mypy
64     $(PYTHON) -m mypy src/
65
66 .PHONY: check
67 check: ## Alles: lint + typecheck + test
68     $(PYTHON) -m ruff check .
69     $(PYTHON) -m ruff format --check .
70     $(PYTHON) -m mypy src/
71     $(PYTHON) -m pytest

```

Listing 3.2: Vollständiges Makefile

Listing 3.2 zeigt die wichtigsten Targets. Die Konvention ist: `make <target>` – und das Makefile sucht sich die richtigen Befehle zusammen.

3.4 Python-Environment

Definition

venv (Virtual Environment) Ein *virtuelles Python-Environment* ist ein isoliertes Verzeichnis, in dem Python-Pakete installiert werden, ohne das System-Python zu beeinflussen. In diesem Projekt wird statt Poetry (das viele zusätzliche Komplexität mitbringt) direkt `python -m venv .venv` und `pip install` verwendet – so funktioniert das Setup auf jeder Maschine ohne spezielle Tools.

Beispiel

Setup-Sequenz

```

1 # 1. Virtuelles Environment anlegen
2 python3 -m venv .venv
3
4 # 2. pip aktualisieren
5 .venv/bin/pip install --quiet --upgrade pip
6
7 # 3. Projekt installieren (inkl. dev-deps)
8 .venv/bin/pip install --quiet -e .
9
10 # 4. Tests laufen lassen
11 .venv/bin/pytest

```

Die wichtigsten Python-Pakete sind in Tabelle 3.1 zusammengefasst.

Paket	Version	Zweck
polars	≥0.20	Datenverarbeitung (schneller als Pandas)
pandas	≥2.2	Datenverarbeitung (Kompatibilität)
numpy	≥1.26	Numerik
sqlalchemy	≥2.0	ORM
alembic	≥1.13	Schema-Migrationen
psycopg[binary]	≥3.1	PostgreSQL-Treiber
httpx	≥0.27	HTTP-Client (Vendor-Adapter)
tenacity	≥8.2	Retries
structlog	≥24.1	Logging
pydantic	≥2.6	Datenmodelle
pydantic-settings	≥2.2	Config aus .env
prefect	≥2.16	Workflow-Orchestrierung
tabulate	≥0.9	Formatierte Tabellen im Terminal
pytest	≥8.0	Test-Runner
ruff	≥0.3	Linten + Formatter
mypy	≥1.8	Type-Checker

Table 3.1: Wichtigste Python-Abhängigkeiten

3.5 Bun und TypeScript

Bun ist die bevorzugte Runtime für das TUI (Kapitel 8). Es vereint Paketmanager, Test-Runner und Bundler in einem einzigen, sehr schnellen Binary.

```

1 # Installation (einmalig pro Maschine)
2 curl -fsSL https://bun.sh/install | bash
3 source ~/.bashrc
4
5 # Im tui/-Verzeichnis
6 cd tui
7 bun install                # Installiert Dependencies
8 bun run start              # Startet TUI
9 bun test                   # Lsst alle Tests laufen
10 bun run typecheck         # tsc --noEmit

```

Listing 3.3: Bun-Setup

3.6 Pfade, Ports, Datenverzeichnisse

3.7 Was beim ersten Start passiert

[Boot-Sequenz]

1. make up startet die Docker-Container.
2. Postgres initialisiert sich (5–10 Sekunden) und ist bereit für Verbindungen (pg_isready gibt Exit-Code 0).
3. Adminer wartet auf postgres: service_healthy, startet dann.

Was	Pfad / Port	Wer greift zu
Postgres-Server	localhost:5432	Python, Bun, Adminer, psql
Adminer-Web-UI	localhost:8080	Browser
Prefect-Server-UI	localhost:4200	Browser
PostgreSQL-Daten-Volume	postgres_data	Docker (persistent)
Prefect-Daten-Volume	prefect_data	Docker (persistent)
Python-Pakete	./.venv/	./.venv/bin/python
Bun-Cache	~/.bun/	Bun
TUI-Node-Modules	./tui/node_modules/	Bun
Jupyter-Notebook-Output	./notebooks/	Browser (HTML)

Table 3.2: Pfade und Ports aller Komponenten

4. Prefect-Server startet (langsamer, 10–20 Sekunden).
5. `make migrate` führt alle Alembic-Migrationen aus und erzeugt die 14 Tabellen.
6. `make ingest-<asset_class>` oder `pytest` oder `make tui` können jetzt starten.

3.8 Was als Nächstes?

Mit der Infrastruktur im Griff wenden wir uns dem nächsten zentralen Thema zu: dem *Datenmodell*. Kapitel 4 erklärt die 14 Tabellen, ihre Beziehungen und ihre Indizes.

Chapter 4

Datenmodell

Das Datenmodell ist das *Fundament* des gesamten Projekts. Es definiert, welche Daten gespeichert werden, wie sie zusammenhängen und wie effizient wir sie abfragen können.

4.1 Überblick: Die 14 Tabellen

Tabelle 4.1 gibt einen schnellen Überblick über alle 14 Tabellen. Die vollständigen DDL-Definitionen finden sich in Anhang C.

Tabelle	Zweck	PK	Wichtige FKs
venues	Börsen/Handelsplätze	venue_id	–
vendors	Datenlieferanten	vendor_id	–
instruments	Asset-Master	instrument_id	primary_venue
instrument_identifiers	Cross-Vendor-Mapping	(scheme, value)	instrument_id
trading_calendars	Handelstage je Venue	(venue_id, date)	venue_id
prices_raw	Roh-Preise (vendor-original)	(vendor_id, vendor_symbol, date)	vendor_id
prices_daily	OHLCV (harmonisiert)	(instrument_id, date)	instrument_id, v
corporate_actions	Splits, Dividenden	(instrument_id, ex_date, action_type)	instrument_id
fx_rates_daily	Cross-Currency-FX	(ccy_from, ccy_to, date)	vendor_id
bond_yields	Bond-Yield-Curve	(instrument_id, date)	instrument_id
macro_series	Makro-Stammdaten	series_id	–
macro_observations	Makro-Beobachtungen	(series_id, date)	series_id
dq_runs	DQ-Lauf-Metadaten	run_id	–
dq_findings	DQ-Befunde	(run_id, check_name)	run_id

Table 4.1: Alle 14 Tabellen des deephold_db-Schemas

4.2 ER-Diagramm

Abbildung 4.1 zeigt das vollständige ER-Diagramm.

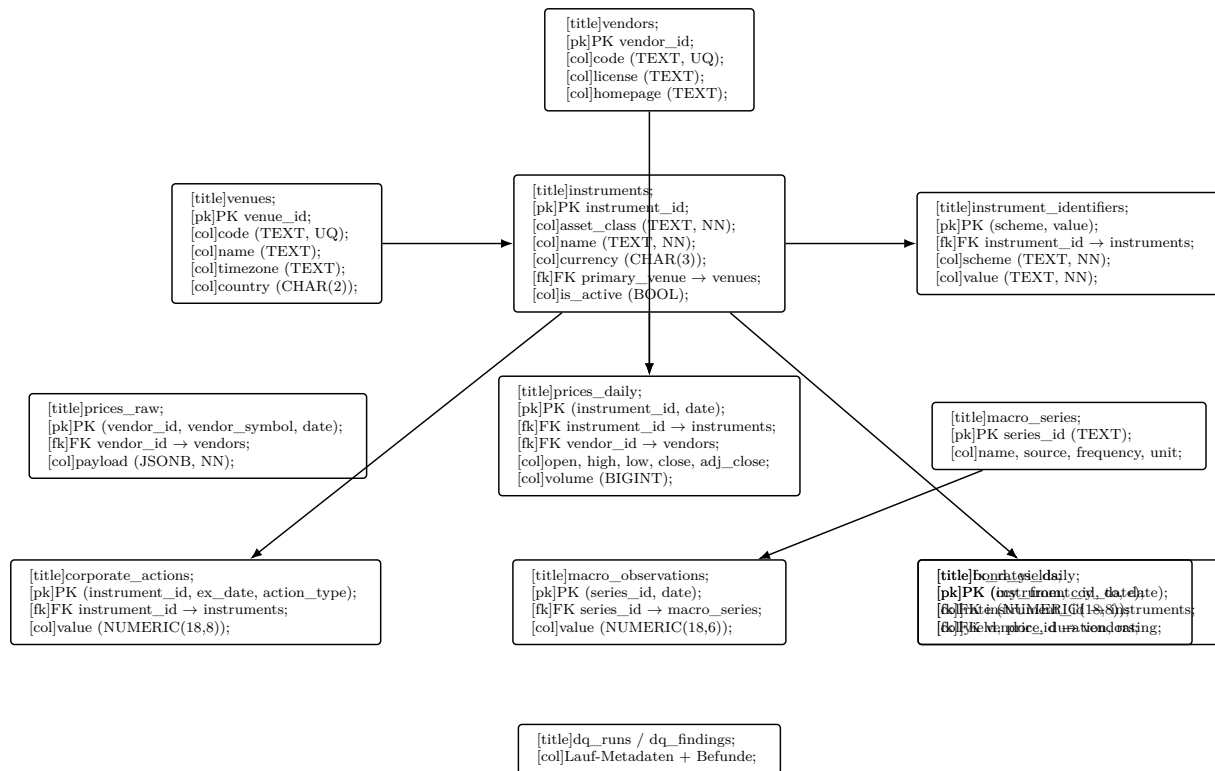


Figure 4.1: Entity-Relationship-Diagramm des deephold_db-Schemas. `instruments` ist der zentrale Asset-Master; `vendors` ist die Quelle aller Preise. Makro- und Equity-Daten sind *parallel* organisiert, nicht in einer Hierarchie.

4.3 Stammdaten-Tabellen

4.3.1 venues

Börsen und Handelsplätze. Beispiele: NASDAQ, NYSE, XETRA, LSE, TYO. Jede Zeile beschreibt einen Handelsplatz eindeutig über den Code (NASDAQ, NYSE, XETRA ...). Die `timezone` ermöglicht später korrekte Datums-Berechnungen über Zeitzonen hinweg.

4.3.2 vendors

Datenlieferanten – die Quellen, aus denen wir Preise beziehen. Beispiele: fred, ecb, yahoo, demo. Die `license`-Spalte speichert die Nutzungsbedingungen (z. B. *public domain*, *CC BY 4.0*, *Yahoo ToS* – *privater Gebrauch*).

4.3.3 instruments

Der Asset-Master – eine zentrale Tabelle, in der jedes gehandelte Wertpapier genau einmal vorkommt: eine Aktie, ein ETF, ein Index, eine Anleihe. `asset_class` ist ein *Diskriminator* (equity, index, bond_gov, ...), der bestimmt, in welchen weiteren Tabellen das Instrument referenziert wird.

4.3.4 instrument_identifiers

Das Cross-Vendor-Mapping. Ein Instrument kann unter verschiedenen IDs bei verschiedenen Vendors bekannt sein:

- Yahoo Finance: AAPL
- ISIN: US0378331005
- Bloomberg: AAPL:US
- FRED: DEXUSEU (für EUR/USD-FX)

instrument_identifiers bildet jede dieser IDs auf die *interne* instrument_id ab. Der zusammengesetzte Primärschlüssel (scheme, value) verhindert Duplikate.

4.4 Preis-Tabellen

4.4.1 prices_raw – Das unveränderte Original

prices_raw speichert die *exakten Roh-Payloads* der Vendors, ohne Interpretation. Jede Zeile enthält:

- vendor_id – welcher Lieferant
- vendor_symbol – dessen nativers Ticker-Symbol
- date – Handelstag
- payload – die vollständige rohe Antwort als JSONB
- ingested_at – Zeitstempel des Pulls

Merke

prices_raw ist der *Append-Only Audit-Trail*. Wir schreiben hinein, ändern aber nie. Selbst wenn die harmonisierte Tabelle prices_daily korrigiert werden muss, ist die Original-Antwort noch da.

4.4.2 prices_daily – Die harmonisierte Zeitreihe

prices_daily ist die *Arbeitstabelle* für alle Preis-basierten Analysen. Sie ist auf instrument_id *normalisiert* (nicht auf das Vendor-Symbol) – dadurch können wir Daten verschiedener Vendors für dasselbe Instrument zusammenführen und vergleichen.

```
1 \d prices_daily
```

Listing 4.1: Auszug aus prices_daily

	Column	Type	Nullable	Default
1				
2				

```

3  --
    -----+-----+-----+-----+
4  instrument_id      | bigint                | not null |
5  date              | date                 | not null |
6  open              | numeric(18,8)        |          |
7  high              | numeric(18,8)        |          |
8  low               | numeric(18,8)        |          |
9  close             | numeric(18,8)        |          |
10 adjusted_close    | numeric(18,8)        |          |
11 volume            | bigint               |          |
12 vendor_id         | integer              |          |
13 Indexes:
14     "prices_daily_pkey" PRIMARY KEY, btree (instrument_id, date)
15     "ix_prices_daily_date" btree (date)
16     "ix_prices_daily_instrument_id_date" btree (instrument_id, date)
17 Check constraints:
18     "prices_daily_check" CHECK (instrument_id IS NOT NULL)
19 Foreign-key constraints:
20     "prices_daily_instrument_id_fkey" FOREIGN KEY (instrument_id)
        REFERENCES instruments(instrument_id)
21     "prices_daily_vendor_id_fkey" FOREIGN KEY (vendor_id) REFERENCES
        vendors(vendor_id)

```

Beachten Sie die Indizes: `ix_prices_daily_date` beschleunigt *zeitbasierte* Queries (`WHERE date BETWEEN ...`), der zusammengesetzte Index `ix_prices_daily_instrument_id_date` beschleunigt Queries nach *einem Instrument über die Zeit*.

4.5 Makro-Tabellen

4.5.1 `macro_series` und `macro_observations`

Makro-Daten (Zinssätze, Inflation, Arbeitsmarkt) werden in zwei *getrennte* Tabellen aufgeteilt:

- `macro_series` – Stammdaten: Name, Quelle, Frequenz, Einheit.
- `macro_observations` – die eigentlichen Werte, eine Zeile pro (Serie, Datum).

Diese Aufteilung ist ein klassisches *Star-Schema*: Die `series`-Tabelle ist die *Dimension*, `observations` ist die *Fact*-Tabelle. Sie ermöglicht, eine Serie nachträglich um Metadaten zu ergänzen (z. B. um die Datenquelle zu wechseln), ohne alle Beobachtungen anzufassen.

4.6 FX- und Bond-Tabellen

4.6.1 `fx_rates_daily`

Wechselkurse – eine Zeile pro (`ccy_from`, `ccy_to`, `date`). Die `ccy_from/ccy_to`-Codierung statt einer `rate`-Tabelle mit zwei Zeilen pro Paar spart Speicher und macht Queries schneller.

4.6.2 bond_yields

Bond-spezifische Daten – die allgemeine `prices_daily`-Tabelle enthält keine Yield-Curve-Daten. Hier werden zusätzlich `yield`, `price`, `duration` und `rating` gespeichert.

4.7 DQ-Tabellen

4.7.1 dq_runs und dq_findings

Data-Quality-Tabellen. `dq_runs` protokolliert jeden Lauf eines DQ-Checks (z. B. *OHLC-Konsistenz* oder *no negative prices*), `dq_findings` die einzelnen Befunde. Diese Tabellen sind derzeit nur *Skelette* – die konkreten Checks werden in einem späteren Kapitel erläutert.

4.8 SQLAlchemy-Definitionen

Das Schema wird in Python deklariert. Listing ?? zeigt einen Auszug.

```

1  class Instrument(Base):
2      __tablename__ = "instruments"
3      instrument_id: Mapped[int] = mapped_column(BigInteger, primary_key=
4          True, autoincrement=True)
5      asset_class: Mapped[str] = mapped_column(Text, nullable=False)
6      name: Mapped[str] = mapped_column(Text, nullable=False)
7      currency: Mapped[str | None] = mapped_column(CHAR(3))
8      primary_venue: Mapped[int | None] = mapped_column(Integer,
9          ForeignKey("venues.venue_id"))
10     is_active: Mapped[bool] = mapped_column(Boolean, server_default=func
11         .true())
12     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True)
13         , server_default=func.now())
14
15 class PricesDaily(Base):
16     __tablename__ = "prices_daily"
17     instrument_id: Mapped[int] = mapped_column(
18         BigInteger, ForeignKey("instruments.instrument_id"), primary_key
19         =True
20     )
21     date: Mapped[date] = mapped_column(Date, primary_key=True)
22     open: Mapped[Decimal | None] = mapped_column(Numeric(18, 8))
23     high: Mapped[Decimal | None] = mapped_column(Numeric(18, 8))
24     low: Mapped[Decimal | None] = mapped_column(Numeric(18, 8))
25     close: Mapped[Decimal | None] = mapped_column(Numeric(18, 8))
26     adjusted_close: Mapped[Decimal | None] = mapped_column(Numeric(18,
27         8))
28     volume: Mapped[int | None] = mapped_column(BigInteger)
29     vendor_id: Mapped[int | None] = mapped_column(Integer, ForeignKey("
30         vendors.vendor_id"))
31     __table_args__ = (

```

```
25         Index("ix_prices_daily_date", "date"),
26         Index("ix_prices_daily_instrument_id_date", "instrument_id", "
           date"),
27     )
```

Listing 4.2: Auszug aus `src/deephold_db/db/models.py`

Listing ?? zeigt die zentrale *Declarative*-Syntax von SQLAlchemy 2.x. Die `Mapped[]`-Annotation gibt Typ + Optionalität an, der `mapped_column()`-Aufruf definiert die Datenbank-Repräsentation.

4.9 Was als Nächstes?

Mit dem Schema im Hinterkopf wenden wir uns den *Vendor-Adaptern* zu – den Komponenten, die Daten aus FRED, ECB und Yahoo Finance in dieses Schema einfüllen. Kapitel 5.

Chapter 5

Vendor-Adapter

Vendor-Adapter sind die Komponenten, die mit den externen Datenquellen sprechen. Jeder Adapter hat die gleiche Schnittstelle – `fetch(symbol, start, end)` und `healthcheck()` – aber unterschiedliche Implementierungen je nach API-Protokoll der Datenquelle.

5.1 Das Vendor-Interface

Definition

Vendor-Adapter Ein *Vendor-Adapter* ist eine Python-Klasse, die eine bestimmte externe Datenquelle (z. B. FRED) abstrahiert. Sie bietet eine *einheitliche* Schnittstelle, damit der Rest des Systems nicht wissen muss, ob die Daten von einer REST-API, einer SDMX-Quelle oder einer CSV-Datei kommen.

Listing 5.1 zeigt die *Abstrakte-Basisklasse*, von der alle konkreten Adapter erben.

```
1 from abc import ABC, abstractmethod
2 from datetime import date
3 import polars as pl
4
5 class Vendor(ABC):
6     """Base class for all vendor adapters."""
7     code: str          # e.g. "fred", "ecb", "yahoo"
8     base_url: str      # Vendor's API base URL
9
10    @abstractmethod
11    def fetch(self, symbol: str, start: date, end: date) -> pl.DataFrame
12        :
13        """Fetch raw data for `symbol` in [start, end] inclusive."""
14        raise NotImplementedError
15
16    @abstractmethod
17    def healthcheck(self) -> bool:
18        """Return True if the vendor is reachable."""
19        raise NotImplementedError
```

Listing 5.1: Basisklasse aller Vendor-Adapter

Die Wahl von `polars.DataFrame` als Rückgabetyt ist bewusst: Polars ist schnell, hat Lazy-Evaluation und ist vollständig typisiert. Die Adapter sind dünn – sie übersetzen JSON oder XML in ein `DataFrame` – und die schwere Arbeit (Statistiken, Resampling, Joins) passiert downstream in dedizierten Analytics-Modulen (siehe Kapitel 10).

5.2 FRED – Federal Reserve Economic Data

Definition

FRED *FRED* (Federal Reserve Economic Data) ist die Wirtschaftsdatenbank der US-Notenbank Federal Reserve. Sie enthält über 800.000 Zeitreihen zu Zinssätzen, Inflation, Beschäftigung, BIP, Geldmenge und vielem mehr. Die Daten sind *public domain* und über eine kostenlose REST-API zugänglich.

Definition

API-Key Ein *API-Key* ist ein geheimer Token, der den *API-Aufrufer* identifiziert. FRED verlangt einen Key, den man kostenlos unter https://fred.stlouisfed.org/docs/api/api_key.html beantragen kann. Der Key wird in der Datei `.env` (nie im Quellcode!) gespeichert.

FRED liefert Zeitreihen über die Endpoints `/fred/series/observations` (für eine einzelne Zeitreihe) und `/fred/series` (für Metadaten). Listing 5.2 zeigt, wie ein typischer Fetch aussieht.

```

1  import httpx
2  from deephold_db.utils.rate_limit import limit
3  from deephold_db.utils.retry import http_retry
4  from deephold_db.utils.config import get_settings
5
6  class FredVendor(Vendor):
7      code = "fred"
8      base_url = "https://api.stlouisfed.org/fred"
9
10     def __init__(self, api_key: str | None = None) -> None:
11         self.api_key = api_key or get_settings().fred_api_key
12         if not self.api_key:
13             raise ValueError("FRED_API_KEY is required (set in .env)")
14
15     @http_retry()
16     def _get(self, path: str, params: dict) -> dict:
17         with limit(self.code, 120, 30): # 120 req/min, burst 30
18             q = {**params, "api_key": self.api_key, "file_type": "json"}
19             r = httpx.get(f"{self.base_url}{path}", params=q, timeout
20                         =30.0)

```

```

20         r.raise_for_status()
21         return r.json()
22
23     def healthcheck(self) -> bool:
24         try:
25             data = self._get("/releases", {"limit": 1})
26             return "releases" in data
27         except Exception:
28             return False
29
30     def fetch(self, symbol: str, start, end) -> pl.DataFrame:
31         data = self._get(
32             "/series/observations",
33             {"series_id": symbol, "observation_start": start.isoformat()
34             ,
35             "observation_end": end.isoformat()},
36         )
37         rows = [{"date": obs["date"], "value": _parse_value(obs["value"]
38             )}]
39         for obs in data.get("observations", []):
40             rows.append({"date": obs["date"], "value": _parse_value(obs["value"]
41             )})
42         return pl.DataFrame(rows, schema={"date": pl.Date, "value": pl.
43             Float64})

```

Listing 5.2: FRED-Vendor: Healthcheck und Fetch

Beachten Sie:

- `@http_retry()` – dekoriert die Methode mit automatischen Retries bei Netzwerkfehlern (3 Versuche, 5s/30s/120s Backoff).
- `with limit(...)` – Token-Bucket, das 120 req/min erlaubt (FREDs Limit) mit Burst 30.
- `_parse_value` – FRED verwendet "." für fehlende Werte; das wird zu NaN konvertiert.

5.3 ECB – European Central Bank SDMX

Definition

SDMX (Statistical Data and Metadata Exchange) *SDMX* ist ein ISO-Standard für den Austausch statistischer Daten. Große Institutionen wie EZB, OECD, IWF und Eurostat veröffentlichen ihre Daten über SDMX-Endpunkte. Das Format hat zwei Häufigkeiten:

- **SDMX-ML** (XML-basiert) – der offizielle Standard
- **SDMX-JSON** (JSON-basiert) – moderner, einfacher zu parsen

Dieses Projekt verwendet die *CSV*-Variante (über `format=csvdata`), weil sie sich am einfachsten in Polars laden lässt.

Beispiel

ECB SDMX URL-Struktur

```

1 # EXR = Exchange Rates
2 # D.USD.EUR.SP00.A = Daily, USD, EUR, Spot, Average
3 https://data-api.ecb.europa.eu/service/data/EXR/D.USD.EUR.SP00.A?
   startPeriod=2024-01-01&endPeriod=2024-12-31&format=csvdata

```

Die ECB-API ist im Gegensatz zu FRED *anonym* – kein API-Key nötig. Listing 5.3 zeigt den ECB-Vendor.

```

1 class ECBVendor(Vendor):
2     code = "ecb"
3
4     def __init__(self, base_url: str | None = None) -> None:
5         self.base_url = (base_url or get_settings().ecb_sdmx_base).
6             rstrip("/")
7
8     @http_retry()
9     def _get_csv(self, path: str, params: dict) -> str:
10         with limit(self.code, 60, 10):
11             q = {**params, "format": "csvdata"}
12             r = httpx.get(f"{self.base_url}{path}", params=q, timeout
13                 =30.0)
14             r.raise_for_status()
15             return r.text
16
17     def fetch(self, symbol, start, end) -> pl.DataFrame:
18         flow, key = self._resolve_symbol(symbol)
19         csv_text = self._get_csv(
20             f"/data/{flow}/{key}",
21             {"startPeriod": start.isoformat(), "endPeriod": end.
22                 isoformat()},
23         )
24         return _parse_ecb_csv(csv_text)

```

Listing 5.3: ECB-Vendor: Parse SDMX CSV

Die SDMX-Daten haben *zwei verschiedene Datumsformate*: Tagesdaten verwenden YYYY-MM-DD (z. B. 2024-06-03), Monatsdaten verwenden YYYY-MM (z. B. 2024-01). Der Parser muss beide Formen handhaben.

```

1 def _parse_ecb_csv(csv_text: str) -> pl.DataFrame:
2     # Disables auto-type-inference (which would mis-parse "2024-01" as
3     # broken date)
4     df = pl.read_csv(
5         csv_text.encode("utf-8"),
6         columns=["TIME_PERIOD", "OBS_VALUE"],

```



```

6         dtypes={"TIME_PERIOD": pl.Utf8, "OBS_VALUE": pl.Float64},
7     )
8     if df.is_empty():
9         return pl.DataFrame(schema={"date": pl.Date, "value": pl.Float64}
10                                })
11
12     # coalesce: first try full date, fall back to year-month
13     df = df.with_columns(
14         pl.coalesce(
15             pl.col("TIME_PERIOD").str.strptime(pl.Date, format="%Y-%m-%d",
16                                                  strict=False),
17             pl.col("TIME_PERIOD").str.strptime(pl.Date, format="%Y-%m",
18                                                  strict=False),
19         ).alias("date")
20     )
21     return df.select(
22         pl.col("date"),
23         pl.col("OBS_VALUE").cast(pl.Float64, strict=False).alias("value")
24     )

```

Listing 5.4: ECB-CSV-Parser: Beide Datumsformate

5.4 Yahoo Finance (via yfinance)

Definition

yfinance *yfinance* ist eine Python-Bibliothek, die *inoffizielle* Yahoo-Finance-API verwendet, um historische Aktien-, ETF- und Index-Daten abzurufen. Es ist *nicht* offiziell von Yahoo unterstützt – die Bibliothek scraped und parst die Yahoo-Website. Daher kann sie bei Yahoo-Änderungen kaputt gehen, und die Yahoo *terms of service* (ToS) erlauben *keine* Weiterverteilung der Daten. Dieses Projekt verwendet *yfinance* daher nur für *private* Forschungszwecke.

Beispiel

yfinance API – sehr einfach

```

1 import yfinance as yf
2 t = yf.Ticker("AAPL")
3 df = t.history(start="2024-01-01", end="2024-12-31", auto_adjust=
4               False)
5 # df hat Spalten: Open, High, Low, Close, Volume, Dividends, Stock
6   Splits
7 # Index: DatetimeIndex mit TZ (z.B. America/New_York)

```

```

1 import yfinance as yf

```

```

2 import pandas as pd
3
4 class YahooVendor(Vendor):
5     code = "yahoo"
6
7     def fetch(self, symbol, start, end) -> pl.DataFrame:
8         with limit(self.code, 30, 5): # 30 req/min, burst 5
9             t = yf.Ticker(symbol)
10            pdf = t.history(
11                start=start.isoformat(),
12                end=(end + timedelta(days=1)).isoformat(), # yf end is
                    exclusive
13                auto_adjust=False, # we need both close and adj_close
14                actions=False, # splits/dividends go to
                    corporate_actions
15            )
16            if pdf.empty:
17                return pl.DataFrame(schema=_YAHOO_SCHEMA)
18            return _yahoo_to_polars(pdf)
19
20    def fetch_many(self, symbols: list[str], start, end) -> dict[str, pl
        .DataFrame]:
21        """Bulk fetch in a single yfinance call -- much faster."""
22        with limit(self.code, 30, 5):
23            pdf = yf.download(
24                tickers=" ".join(symbols),
25                start=start.isoformat(),
26                end=(end + timedelta(days=1)).isoformat(),
27                auto_adjust=False,
28                actions=False,
29                group_by="ticker",
30                threads=True,
31                progress=False,
32            )
33            # ... handle MultiIndex columns for multiple tickers

```

Listing 5.5: Yahoo-Vendor: Single und Bulk Fetch

Listing 5.5 zeigt den Yahoo-Vendor. Zwei Dinge sind erwähnenswert:

1. **Bulk-Methode:** `fetch_many` verwendet `yf.download` mit `threads=True` – dies holt *alle* Symbole in einem einzigen API-Call. Für 36 Symbole benötigt es nur 12 Sekunden, statt 36 einzelne Calls.
2. **Pandas-zu-Polars-Bridge:** `yfinance` liefert ein `pandas.DataFrame` mit TZ-aware-DatetimeIndex und nullable Integers (Int64 statt int64). Wir konvertieren explizit zu pl-kompatiblen Typen (int64) und parsen die TZ-Datums in Polars.

5.5 Rate-Limiting und Retry

Definition

Exponential Backoff *Exponential Backoff* ist eine Retry-Strategie, bei der die Wartezeit zwischen Versuchen *exponentiell wächst*: 5s, dann 30s, dann 120s. Dies gibt einem überlasteten Server Zeit, sich zu erholen, ohne dass der Client ihn weiter bombardiert.

Listing 5.6 zeigt die zentralen Werkzeuge für robustes HTTP.

```

1  import httpx
2  import time
3  from tenacity import retry, stop_after_attempt, wait_chain, wait_fixed
4
5  # Token-Bucket f r Rate-Limiting
6  class TokenBucket:
7      def __init__(self, rate_per_min: int, burst: int):
8          self.rate = rate_per_min / 60.0 # tokens per second
9          self.burst = burst
10         self.tokens = float(burst)
11         self.last = time.monotonic()
12
13     def take(self, n: int = 1) -> None:
14         # Wenn nicht genug Tokens: blockiere bis genug da sind
15         now = time.monotonic()
16         elapsed = now - self.last
17         self.last = now
18         self.tokens = min(self.burst, self.tokens + elapsed * self.rate)
19         if self.tokens < n:
20             time.sleep((n - self.tokens) / self.rate)
21             self.tokens = 0
22         else:
23             self.tokens -= n
24
25 # Retry-Decorator mit 5s/30s/120s Backoff
26 def http_retry():
27     return retry(
28         reraise=True,
29         stop=stop_after_attempt(3),
30         wait=wait_chain(wait_fixed(5), wait_fixed(30), wait_fixed(120)),
31         retry=retry_if_exception_type((
32             httpx.TimeoutException, httpx.NetworkError, httpx.
33                 RemoteProtocolError,
34         )),
35     )

```

Listing 5.6: Rate-Limiting und Retry-Logik

Jeder Vendor-Adapter verwendet `limit(code, rate, burst)` und `@http_retry()`, um die spezifischen

Anforderungen der Quelle zu erfüllen. Tabelle 5.1 zeigt die Limits pro Vendor.

Vendor	Rate	Burst	Retry-Backoff
FRED	120 req/min	30	5s/30s/120s
ECB	60 req/min	10	5s/30s/120s
Yahoo	30 req/min	5	5s/30s/120s

Table 5.1: Rate-Limits und Burst pro Vendor. Diese sind in `src/deephold_db/vendors/<name>.py` konfiguriert.

5.6 Was als Nächstes?

Die Vendor-Adapter sind die Datenquellen; die ETL-Pipeline (Kapitel 6) zeigt, wie diese Daten in unsere Datenbank gelangen und für Analysen verfügbar gemacht werden.

Chapter 6

ETL-Pipeline

Die *ETL-Pipeline* – Extract, Transform, Load – ist das *Rückgrat* des Projekts. Sie ist verantwortlich dafür, dass die Daten der Vendor-Adapter tatsächlich in die Datenbank gelangen und für Analysen verfügbar werden.

6.1 Überblick

Definition

ETL *ETL* steht für:

- **Extract** – Daten aus den Quellen (FRED, ECB, Yahoo) abrufen.
- **Transform** – Daten in das richtige Format und Schema bringen.
- **Load** – Daten in die Datenbank schreiben.

Abbildung 6.1 zeigt den ETL-Flow im Detail.

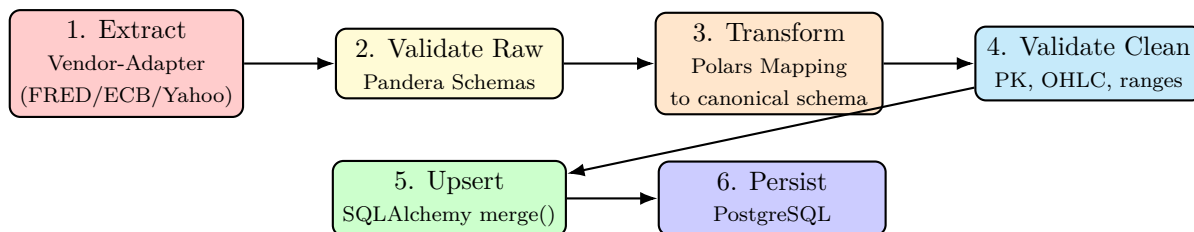


Figure 6.1: Der ETL-Flow: Sechs Schritte, jeweils mit klarer Verantwortung. Im aktuellen Projekt sind die Validate-Schritte vorbereitet, aber noch nicht vollständig implementiert (geplant für eine DQ-Iteration).

6.2 Der Seeder-Pattern

Listing 6.1 zeigt das Standard-Pattern zum *Seed* (Initial-Befüllung) einer Tabelle. Es wird für `vendors`, `instruments` und `prices_daily` verwendet.

```
1 def upsert_instrument(asset_class, name, currency, symbol):
2     """Idempotent: creates the instrument if not in DB, else returns id.
    """
```

```

3     with session_scope() as s:
4         existing = (
5             s.query(Instrument)
6             .join(InstrumentIdentifier,
7                 InstrumentIdentifier.instrument_id == Instrument.
8                     instrument_id)
9             .filter(
10                 InstrumentIdentifier.scheme == "YAHOO",
11                 InstrumentIdentifier.value == symbol,
12             )
13             .one_or_none()
14         )
15         if existing is None:
16             inst = Instrument(asset_class=asset_class, name=name,
17                             currency=currency)
18             s.add(inst)
19             s.flush() # gets us inst.instrument_id
20             s.add(InstrumentIdentifier(
21                 instrument_id=inst.instrument_id,
22                 scheme="YAHOO",
23                 value=symbol,
24             ))
25             s.flush()
26             return inst.instrument_id
27         return existing.instrument_id

```

Listing 6.1: Seeder-Pattern: lookup-or-create

Die Schlüsseleigenschaft ist *Idempotenz*: Wenn der Seeder zweimal läuft, werden keine Duplikate angelegt. `merge()` allein reicht dafür nicht aus – wenn das Objekt keine primary key hat, versucht SQLAlchemy ein INSERT statt eines UPDATE. Die explizite SELECT-Suche im Voraus ist robuster.

6.3 Das `query_db.py`-Skript

Das `query_db.py`-Skript ist die zentrale CLI für Daten-Operationen. Es kann:

- die Datenbank mit Demo-Daten füllen (falls leer),
- echte Daten von FRED, ECB und Yahoo holen,
- die Daten in tabellarischer Form im Terminal anzeigen.

Listing 6.2 zeigt einen vereinfachten Auszug.

```

1 def main() -> int:
2     settings = get_settings()
3     configure_logging(settings.log_level)
4
5     with session_scope() as s:

```

```

6         n_obs = s.execute(select(func.count()).select_from(
            MacroObservation)).scalar() or 0
7
8     if n_obs == 0 and not args.no_seed:
9         # No data: try ECB + FRED in sequence
10        if settings.fred_api_key:
11            print("seeding real FRED data...")
12            _seed_from_fred(settings.fred_api_key)
13        print("seeding real ECB data (no API key required)...")
14        _seed_from_ecb()
15        print("seeding real Yahoo Finance data (AAPL, MSFT, ^GSPC)...")
16        _seed_from_yahoo()
17
18    # Display all series with their latest values
19    series_list = _fetch_series(args.series)
20    _print_summary(series_list, tail=args.tail)
21    return 0

```

Listing 6.2: Auszug aus scripts/query_db.py

Beachten Sie das *Reihenfolge*-Pattern:

1. Prüfe, ob die DB bereits Daten hat ($n_obs > 0$).
2. Falls leer: Versuche ECB (kein Key) und FRED (falls Key gesetzt). Fällt auf Demo-Daten zurück, wenn beide fehlschlagen.
3. Lese alle Serien und zeige sie formatiert an.

6.4 Bulk-Seeding mit seed_paper_data.py

Für die AEGIS-Paper-Replikation (Kapitel 10) haben wir ein eigenes Skript gebaut, das 36 Symbole (31 Aktien + 5 Benchmarks) in einem einzigen Aufruf von yfinance lädt.

```

1 # Einmaliger Aufruf
2 .venv/bin/python scripts/seed_paper_data.py
3
4 # Mit benutzerdefiniertem Zeitraum
5 .venv/bin/python scripts/seed_paper_data.py --start 2006-01-01
6
7 # Ohne Benchmarks
8 .venv/bin/python scripts/seed_paper_data.py --no-benchmarks

```

Listing 6.3: Bulk-Seed-Skript aufrufen

Das Skript nutzt die fetch_many-Bulk-Methode des Yahoo-Vendors (Listing 5.5) – 36 Symbole werden in einem einzigen yf.download-Aufruf geholt, was 12 Sekunden statt 36 einzelner Calls benötigt. Insgesamt werden 185.220 Zeilen in die prices_daily-Tabelle geschrieben.

6.5 DQ-Checks (Vorbereitung)

Das Projekt hat Skelette für Data-Quality-Checks, die in einer späteren Iteration implementiert werden sollen. Die DQ-Tabellen (`dq_runs`, `dq_findings`) existieren bereits.

Folgende Checks sind geplant:

- **PK-Eindeutigkeit:** (`instrument_id`, `date`) darf nicht doppelt vorkommen.
- **OHLC-Konsistenz:** $\text{low} \leq \text{open}$, $\text{close} \leq \text{high}$ (keine *inverted candles*).
- **Keine negativen Preise/Volumes.**
- **Adjusted-Close-Konsistenz:** Wenn `corporate_actions` verfügbar ist, muss `adjusted_close` das Produkt aus `close` und allen Splits/Dividenden sein.
- **Lücken-Detection:** Mehr als 5 aufeinanderfolgende Handelstage ohne Eintrag – ein *DQ-Finding*.
- **Ausreißer-Detection:** Tagesreturn $> 6\sigma$ gegenüber rollendem 252-Tage-Fenster – *flag*, nicht löschen.
- **Currency-Coverage:** Jedes `instrument` hat `currency` gesetzt.
- **Vendor-Coverage:** Jede Zeile in `prices_daily` hat `vendor_id` gesetzt.

6.6 Idempotenz

Merke

Alle Seeder sind *idempotent*. Sie können beliebig oft ausgeführt werden, ohne Duplikate zu erzeugen. Dies ist entscheidend für die Inkrementelle-Aktualisierung: ein täglicher Cron-Job ruft den Seeder auf, und nur neue Daten werden tatsächlich geschrieben.

Listing 6.4 zeigt das zentrale Idempotenz-Pattern.

```

1 def upsert_prices(instrument_id, vendor_id, df) -> int:
2     if df.is_empty():
3         return 0
4     rows = 0
5     with session_scope() as s:
6         for r in df.iter_rows(named=True):
7             s.merge(PricesDaily(
8                 instrument_id=instrument_id,
9                 date=r["date"],
10                open=r.get("open"),
11                high=r.get("high"),
12                low=r.get("low"),
13                close=r["close"],
14                adjusted_close=r.get("adjusted_close"),
15                volume=r.get("volume"),
16                vendor_id=vendor_id,
```



```
17         ))
18         rows += 1
19     return rows
```

Listing 6.4: Idempotenz-Pattern: SELECT vor INSERT

6.7 Failure-Isolation

Definition

Failure-Isolation *Failure-Isolation* bedeutet, dass der Fehler eines Vendors *nicht* den gesamten Pipeline-Lauf abbricht. Wenn Yahoo gerade ein Problem hat, sollen ECB und FRED trotzdem seeden.

Das Skript `query_db.py` setzt das um, indem jeder Vendor-Seeder in einem separaten `session_scope` läuft. Ein Fehler in einem Vendor wird geloggt, der nächste Vendor läuft trotzdem.

6.8 Was als Nächstes?

Die ETL-Pipeline produziert Daten in `prices_daily` und `macro_observations`. Im nächsten Kapitel sehen wir, wie diese Daten in einem *Jupyter-Notebook* visualisiert werden können.

Chapter 7

Notebook-Generator

Das *Jupyter-Notebook* ist die komfortabelste Form, um Finanzmarktdaten zu visualisieren. Es ermöglicht interaktive Charts, schrittweises Erforschen und einfaches Teilen der Ergebnisse als HTML.

7.1 Überblick: Pipeline

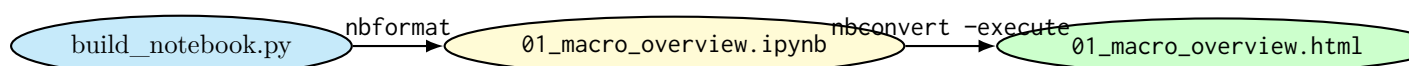


Figure 7.1: Build-Pipeline für das Notebook. `build_notebook.py` erzeugt die `.ipynb` programmatisch aus Cell-Definitionen. `jupyter nbconvert` führt die Cells aus und exportiert HTML.

7.2 Warum programmatisch generieren?

Achtung

Notebook-Ausführungs-Ausgaben veralten schnell. Wenn ein Notebook committet wird, sind die eingebetteten Plots und Tabellen für immer eingefroren – egal, wie sich die Daten dahinter ändern. Das führt zu stale-notebooks, in denen der Leser nicht erkennt, dass die Daten veraltet sind.

Die Lösung: *Trennung* zwischen Quellcode (Cell-Definitionen in Python) und *Output* (gerendertes Notebook + HTML). Die `.ipynb`-Datei ist ein *Build-Artefakt*, das jederzeit regeneriert werden kann.

7.3 Der Build-Prozess

Listing 7.1 zeigt das Herzstück von `scripts/build_notebook.py`.

```
1 import nbformat as nbf
2
3 CELLS = [
```

```

4     nbf.v4.new_markdown_cell("# deephold_db      Macro Overview"),
5     nbf.v4.new_code_cell("from deephold_db.db import session_scope,
6                           MacroObservation"),
7     nbf.v4.new_code_cell("..."),
8     nbf.v4.new_markdown_cell("## Plots"),
9     nbf.v4.new_code_cell("fig = make_subplots(...)",),
10    nbf.v4.new_code_cell("fig.show()"),
11    nbf.v4.new_markdown_cell("## Summary"),
12    nbf.v4.new_code_cell("summary = pl.DataFrame(rows); print(summary)")
13
14 ]
15
16 def main():
17     nb = nbf.v4.new_notebook()
18     nb.cells = CELLS
19     nb.metadata = {"kernel_spec": {"name": "python3"}}
20     nbf.write(nb, NOTEBOOK_PATH)
21     subprocess.run(["jupyter", "nbconvert", "--execute", "--inplace",
22                     str(NOTEBOOK_PATH)])
23     subprocess.run(["jupyter", "nbconvert", "--to", "html", str(
24                     NOTEBOOK_PATH)])

```

Listing 7.1: Notebook programmatisch erzeugen

Der Build-Prozess ist in vier Schritte:

1. **Cell-Definition:** Python-Liste von `nbf.v4.new_markdown_cell()` und `nbf.v4.new_code_cell()` Objekten.
2. **Notebook schreiben:** `nbf.write()` erzeugt eine valides `.ipynb`-JSON-Datei.
3. **Execute:** `jupyter nbconvert -execute` führt alle Code-Cells aus und bettet die Outputs in die `.ipynb` ein.
4. **Export:** `jupyter nbconvert -to html` erzeugt eine statische HTML-Datei, die ohne Jupyter geöffnet werden kann.

7.4 Plotly 3×2-Grid

Das Notebook zeigt 6 Subplots in einem 3×2-Grid (Abbildung 7.2):

Jeder Subplot hat seine eigenen *Y-Achsen-Skalen*, weil die Größenordnungen fundamental verschieden sind (Zinssätze in %, Aktienkurse in USD, Volumen in Milliarden).

7.5 HTML-Export

Der HTML-Export hat zwei Verwendungszwecke:

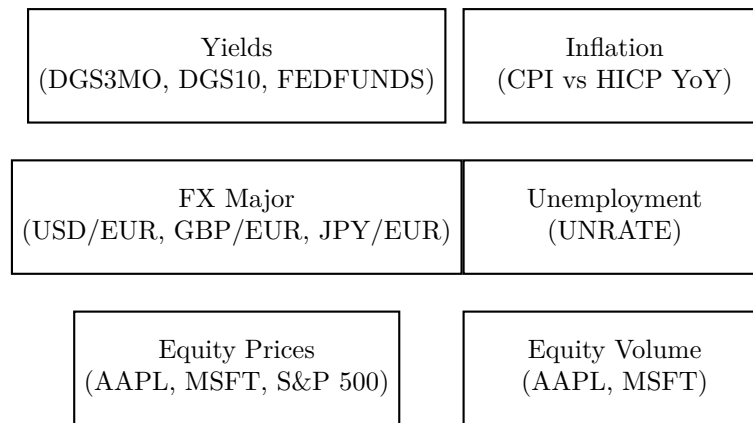


Figure 7.2: 3×2 Plotly-Subplot-Grid im Notebook. Die obere Hälfte zeigt Makro-Daten, die untere Hälfte Equity-Daten.

- **Sharing:** Man kann die HTML-Datei per Mail verschicken oder auf einem Webserver ablegen, ohne dass der Empfänger Jupyter installieren muss.
- **Caching:** Die HTML-Datei ist ein *Snapshot* der Daten zum Build-Zeitpunkt. Wenn die DB später aktualisiert wird, bleibt der HTML-Snapshot konsistent.

Merke

Die HTML-Datei *referenziert* plotly.js von einem CDN. Ohne Internet-Zugang beim Öffnen der HTML-Datei sind die Plots *nicht* sichtbar – die zugrundeliegenden Daten sind aber weiterhin im HTML-Markup eingebettet.

7.6 Was als Nächstes?

Das Notebook ist eine schöne *Passive-View* der Daten. Für *aktive Exploration* – Springen zwischen Serien, Suche, Keyboard-Navigation – brauchen wir etwas anderes: das *OpenTUI-Explorer*, das im nächsten Kapitel vorgestellt wird.

Chapter 8

OpenTUI-Explorer

Der *OpenTUI-Explorer* ist die schnellste und interaktivste Möglichkeit, die gesammelten Daten zu durchsuchen. Er läuft direkt im Terminal, ohne Jupyter, ohne Notebook, ohne Python-Overhead im Hot-Path.

8.1 Was ist OpenTUI?

Definition

OpenTUI OpenTUI (<https://opentui.com>) ist ein von Anomaly (<https://anoma.ly>) entwickeltes TUI-Framework. Es besteht aus:

- einem in *Zig* geschriebenen *Core* (schnell, plattformübergreifend, nativ),
- einer *C-ABI* für Bindings aus jeder Sprache,
- *TypeScript*-Bindings mit React- und Solid.js-Support.

Der Kern nutzt *Yoga* (Facebooks Flexbox-Engine) für Layouts und rendert das Ergebnis direkt in einen Terminal-Buffer.

Merke

OpenTUI ist *die gleiche Engine*, die *opencode* (powered by Anomaly) und *terminal.shop* antreibt. Es befindet sich in aktiver Entwicklung, kann also Breaking Changes enthalten.

8.2 Warum TUI statt GUI?

Aspekt	TUI	GUI (z. B. Web-App)
Startup-Zeit	<100 ms (Bun)	>1 s (Browser)
Speicherverbrauch	~50 MB	~300 MB
Tastatur-Effizienz	<i>Vim-style</i>	Maus
SSH-/Server-tauglich	ja	nur mit X-Forwarding
Visueller Reiz	niedrig	hoch
Lernkurve	hoch (Tasten lernen)	niedrig (klicken)

Figure 8.1: TUI vs. GUI

Für die tägliche Exploration von Finanzmarktdaten – springen zwischen 12 Serien, monatliche Updates prüfen, Daten vergleichen – ist eine TUI das richtige Werkzeug.

8.3 Architektur des TUI-Layers

Das TUI ist ein *eigenständiger TypeScript-Subtree* im Projekt-Root:

```

1  tui/
2      package.json          # Bun + React 19 + OpenTUI 0.4
3      tsconfig.json
4      bun.lock
5      src/
6          index.tsx          # Entry: createCliRenderer +
        createRoot
7          App.tsx            # Root-Komponente (Layout, State,
        Keyboard)
8          components/
9              SeriesList.tsx  # Linke Seite: 12 Serien
10             SeriesDetail.tsx # Rechte Seite: Stats + Tail
11         data/
12             db.ts           # pg-Pool
13             queries.ts      # 4 SQL-Queries
14             types.ts        # TypeScript-Typen
15         tests/              # bun test (Unit + Integration)
16         scripts/check-db.ts # DB-Connectivity-Test

```

Listing 8.1: TUI-Verzeichnisstruktur

8.4 Die Komponenten

8.4.1 Entry Point: index.tsx

```

1  import { createCliRenderer } from "@opentui/core";
2  import { createRoot } from "@opentui/react";
3  import { App } from "../App";
4  import { closePool } from "../data/db";
5
6  const renderer = await createCliRenderer({ exitOnCtrlC: true });
7  const root = createRoot(renderer);
8  root.render(<App />);

```

Listing 8.2: TUI-Entry-Point

Der Entry-Point ist minimal:

1. `createCliRenderer()` – startet den OpenTUI-Renderer, der den Terminal-Buffer verwaltet.
2. `createRoot(renderer)` – erzeugt einen React-Root, der direkt in den OpenTUI-Buffer rendert.
3. `root.render(<App />)` – mountet die App-Komponente.

8.4.2 Root: App.tsx

Die Root-Komponente verwaltet drei Dinge: *State* (welche Serien sind in der DB, welche ist selektiert), *Keyboard-Handling* und das *Layout* (2-Spalten).

```

1  function App() {
2    const renderer = useRenderer();
3    const [series, setSeries] = useState<SeriesRow[]>([]);
4    const [selectedIndex, setSelectedIndex] = useState(0);
5
6    useEffect(() => {
7      listSeries().then(setSeries).catch(/* ... */);
8    }, []);
9
10   useKeyboard((key) => {
11     if (key.name === "q") renderer.destroy();
12     if (key.name === "r") setRefreshToken(t => t + 1);
13     if (key.name === "up" || key.name === "k") setSelectedIndex(i =>
14       Math.max(0, i - 1));
15     if (key.name === "down" || key.name === "j") setSelectedIndex(i =>
16       Math.min(series.length - 1, i + 1));
17   });
18
19   return (
20     <box flexDirection="row">
21       <SeriesList series={series} selectedIndex={selectedIndex} />
22       {series[selectedIndex] && <SeriesDetail series={series[
23         selectedIndex]} />}
24     </box>
25   );
26 }

```

Listing 8.3: TUI-App: State + Keyboard

8.4.3 SeriesList – Linke Spalte

Die SeriesList-Komponente rendert alle geladenen Serien als *scrollbare* Liste. Jede Zeile enthält:

- **Badge:** M für Macro (FRED/ECB), E für Equity (Yahoo).
- **Series-ID:** z. B. FRED:DGS3MO oder ECB:EXR:USD.EUR.SP00.A.
- **Vendor-Tag:** fred, ecb, yahoo.
- **Row-Count:** rechtsbündig für schnelles Scannen.

8.4.4 SeriesDetail – Rechte Spalte

Die SeriesDetail-Komponente zeigt für die ausgewählte Serie:

- **Header:** Series-ID + Name.

- **Stats-Panel:** rows, first/last date, unit, source, min/max/mean.
- **Tail-Tabelle:** die letzten 30 Werte, bei Macro als date, value, bei Equity als date, close, adj_close, vol.

8.5 Die SQL-Queries

Das TUI spricht *direkt* mit der PostgreSQL-DB via node-postgres – es gibt keinen Python-Overhead im Hot-Path.

```

1  const LIST_SERIES_SQL = `
2    SELECT 'macro' AS kind, series_id AS id, name, source, frequency, unit
      , COUNT(*)::int AS n
3    FROM macro_observations mo
4    JOIN macro_series ms USING (series_id)
5    GROUP BY series_id, name, source, frequency, unit
6    UNION ALL
7    SELECT 'equity' AS kind, COALESCE(ii.value, i.instrument_id::text) AS
      id,
8      i.name, 'yahoo' AS source, 'D' AS frequency, i.currency AS unit
      , COUNT(*)::int AS n
9    FROM prices_daily pd
10   JOIN instruments i USING (instrument_id)
11   LEFT JOIN instrument_identifiers ii
12     ON ii.instrument_id = i.instrument_id AND ii.scheme = 'YAHOO'
13   GROUP BY i.instrument_id, ii.value, i.name, i.currency
14   ORDER BY 1, 2
15 `;
16
17 const MACRO_TAIL_SQL = `
18   SELECT to_char(date, 'YYYY-MM-DD') AS date, value
19   FROM macro_observations WHERE series_id = $1
20   ORDER BY date DESC LIMIT $2
21 `;
22
23 const EQUITY_TAIL_SQL = `
24   SELECT to_char(pd.date, 'YYYY-MM-DD') AS date, pd.close, pd.
      adjusted_close, pd.volume
25   FROM prices_daily pd
26   JOIN instrument_identifiers ii
27     ON ii.instrument_id = pd.instrument_id AND ii.scheme = 'YAHOO'
28   WHERE ii.value = $1
29   ORDER BY pd.date DESC LIMIT $2
30 `;
31
32 const MACRO_STATS_SQL = `
33   SELECT COUNT(*)::int AS n, to_char(MIN(date), 'YYYY-MM-DD') AS first,
34     to_char(MAX(date), 'YYYY-MM-DD') AS last, MIN(value) AS min,

```

```

35         MAX(value) AS max, AVG(value) AS mean
36     FROM macro_observations WHERE series_id = $1
37 ;

```

Listing 8.4: 4 SQL-Queries im TUI

Beachten Sie:

- **UNION ALL** – eine einzige Query listet Macro- und Equity-Serien in einer Liste.
- `COALESCE(ii.value, i.instrument_id::text)` – für Equity-Serien wird der YAHOO-Ticker (AAPL) statt der numerischen `instrument_id` (1) angezeigt.
- `LEFT JOIN instrument_identifiers` – damit auch Equity-Serien ohne YAHOO-Identifizierer angezeigt würden (was aktuell nicht vorkommt, aber defensiv ist).

8.6 Tastatur-Handling

Taste	Aktion
↑ / k	Selection nach oben
↓ / j	Selection nach unten
g / Home	Zum Anfang
G / End	Zum Ende
r	Re-Query (für Live-Refresh)
q / Ctrl-C	Quit

Table 8.1: Tastatur-Bindings des TUI. Die j/k-Variante ist Vim-style für Entwickler, die diese Tasten gewohnt sind.

```

1  useKeyboard((key) => {
2    if (key.name === "q" || (key.ctrl && key.name === "c")) {
3      renderer.destroy();
4      setTimeout(() => process.exit(0), 50);
5      return;
6    }
7    if (key.name === "r") {
8      setRefreshToken(t => t + 1);
9      return;
10   }
11   if (key.name === "up" || key.name === "k") {
12     setSelectedIndex(i => Math.max(0, i - 1));
13     return;
14   }
15   // ... etc.
16 });

```

Listing 8.5: Keyboard-Handler

8.7 Tests

Das TUI hat 27 Tests in 4 Dateien:

- `src/data/queries.test.ts` (9 Tests) – SQL-Strings, Row-Mapping, Param-Übergabe.
- `src/data/db.test.ts` (3 Tests) – Public-API des `db`-Moduls.
- `src/types/format.test.ts` (8 Tests) – Pure-Funktionen (`fmtNumber`, `fmtPct`).
- `tests/integration.test.ts` (7 Tests) – Echtes Postgres, 12 Series, OHLCV-Validierung.

```
1 cd tui && bun test
2 # 27 pass, 0 fail, 203 expect(), 4 files
```

Listing 8.6: TUI-Tests laufen

8.8 Was als Nächstes?

Die TUI ist die *operative Oberfläche* des Projekts. Das nächste Kapitel zeigt, wie wir die Datenqualität *testen* – nicht nur funktional (geht der Code?), sondern auch inhaltlich (stimmen die Daten?).

Chapter 9

Tests

Tests sind das Sicherheitsnetz des Projekts. Sie prüfen, ob der Code das tut, was er tun soll – und verhindern, dass spätere Änderungen bestehende Funktionalität brechen.

9.1 Test-Philosophie

Merke

Das Projekt folgt der *Pyramide* der Tests:

- **Viele** schnelle *Unit-Tests* (einzelne Funktionen).
- **Wenige** *Integration-Tests* (mehrere Komponenten zusammen).
- **Manuelle** *Checks* für Dinge, die schwer zu automatisieren sind (visuelle TUI-Korrektheit).

9.2 Python-Tests mit pytest

9.2.1 Überblick

```
1 .venv/bin/pytest
2 # 55 passed
```

Listing 9.1: Python-Tests laufen

55 Tests sind in der Suite (Stand: Handbuch-Erstellung). Sie sind in folgende Module aufgeteilt:

- tests/test_config.py – Konfigurationsladen.
- tests/test_models.py – SQLAlchemy-ORM-Registrierung.
- tests/test_fred.py – FRED-Vendor (mit gemocktem HTTP).
- tests/test_ecb.py – ECB-Vendor (mit gemocktem HTTP).
- tests/test_yahoo.py – Yahoo-Vendor (mit gemocktem yfinance).
- tests/analytics/test_metrics.py – Returns + Metriken.

9.2.2 Test-Fixtures

Definition

pytest Fixture Eine *Fixture* ist eine wiederverwendbare Test-Setup-Funktion. Sie wird mit `@pytest.fixture` dekoriert und kann in Tests *injiziert* werden, indem sie als Argument mit dem gleichen Namen wie die Fixture-Funktion deklariert wird.

```
1 @pytest.fixture(autouse=True)
2 def _reset_settings_cache():
3     """Reset the pydantic-settings lru_cache around each test."""
4     get_settings.cache_clear()
5     yield
6     get_settings.cache_clear()
```

Listing 9.2: pytest-Fixture für Settings-Cache-Reset

Diese `autouse=True`-Fixture wird *vor jedem Test* automatisch ausgeführt und stellt sicher, dass der `lru_cache` der `get_settings()`-Funktion geleert ist – sonst würden Test-Pollution und unstabile Ergebnisse entstehen.

9.2.3 Mocking

Definition

Mock Ein *Mock* ist ein gefälschtes Objekt, das das Verhalten eines realen Objekts nachahmt. In Tests verwenden wir Mocks, um externe Abhängigkeiten (HTTP-Server, Datenbanken) zu simulieren, ohne *tatsächlich* mit ihnen zu sprechen – das macht Tests *schnell*, *deterministisch* und *offline lauffähig*.

Listing 9.3 zeigt einen typischen Mock-Test für den FRED-Vendor.

```
1 def test_fred_fetch_pares_response(fake_fred_observations, monkeypatch)
2     :
3     monkeypatch.setenv("FRED_API_KEY", "test-key")
4     v = FredVendor()
5     v._get = MagicMock(return_value=fake_fred_observations)
6     df = v.fetch("DGS3M0", date(2024, 6, 1), date(2024, 6, 30))
7     assert df.height == 3
8     assert df["value"][0] == pytest.approx(5.02)
9     v._get.assert_called_once()
```

Listing 9.3: FRED-Vendor mit gemocktem HTTP

Die Strategie: `v._get` wird durch einen `MagicMock` ersetzt, der *jeden* Aufruf mit dem vorbereiteten Fake-Response beantwortet. So können wir die *Response-Parsing*-Logik isoliert testen, ohne einen echten HTTP-Server zu kontaktieren.

9.3 TypeScript-Tests mit bun test

9.3.1 Überblick

```
1 cd tui && bun test
2 # 27 pass, 0 fail
```

Listing 9.4: TUI-Tests laufen

9.3.2 Mocking in bun test

Bun's Test-Runner hat eine Jest-kompatible API: describe, test, expect, spyOn, mock.

```
1 import { describe, it, expect, spyOn, type Mock } from "bun:test";
2 import * as dbModule from "../db";
3 import { listSeries } from "../queries";
4
5 let spies: Mock<typeof dbModule.query>[] = [];
6
7 function mockQuery<T>(rows: T[]): Mock<typeof dbModule.query> {
8   const spy = spyOn(dbModule, "query") as Mock<typeof dbModule.query>;
9   spy.mockReset();
10  spy.mockResolvedValue(rows as never);
11  spies.push(spy);
12  return spy;
13 }
14
15 afterEach(() => {
16   for (const s of spies) s.mockRestore();
17   spies = [];
18 });
19
20 describe("listSeries", () => {
21   it("calls query() and returns the rows", async () => {
22     const rows: SeriesRow[] = [{...}];
23     const spy = mockQuery(rows);
24     const out = await listSeries();
25     expect(out).toEqual(rows);
26     expect(spy).toHaveBeenCalledTimes(1);
27     const sql = spy.mock.calls[0]?.[0] as string;
28     expect(sql).toContain("UNION ALL");
29   });
30 });
```

Listing 9.5: Mocking der pg-Layer im TUI

Wichtige Pattern:

- spyOn ersetzt eine Methode durch einen Spy, der Aufrufe aufzeichnet.
- mockResolvedValue setzt den Rückgabewert.

- `afterEach` restauriert die Spies, um Test-Pollution zu vermeiden.

9.3.3 Integration-Tests

```

1  beforeAll(async () => {
2    try {
3      await query("SELECT 1::int AS x");
4      dbReachable = true;
5    } catch (e) {
6      console.warn("Postgres unreachable, tests will no-op.");
7    }
8  });
9
10 function guarded(name, fn) {
11   return async () => {
12     if (!dbReachable) {
13       console.warn(`skip ${name}: DB unreachable`);
14       return;
15     }
16     await fn();
17   };
18 }
19
20 it("equity rows carry the YAHOO ticker (AAPL/MSFT/^GSPC)",
21   guarded("equity-tickers", async () => {
22     const rows = await listSeries();
23     const tickers = rows
24       .filter(r => r.kind === "equity")
25       .map(r => r.id)
26       .sort();
27     expect(tickers).toEqual(["AAPL", "MSFT", "^GSPC"]);
28   }));

```

Listing 9.6: Integration-Test mit echtem Postgres

Integration-Tests prüfen das Verhalten gegen die echte PostgreSQL-DB. Sie sind *no-op* (nicht *fail*), wenn die DB nicht erreichbar ist – so bleibt die CI grün, auch wenn Postgres temporär down ist.

9.4 Manuelle Test-Checklist

Manche Dinge sind schwer zu automatisieren – visuelle Korrektheit einer TUI, Tastatur-Feel, Verhalten unter ungewöhnlichen Bedingungen. Dafür gibt es die manuelle Checkliste in `tui/README.md`.

Beispiel**Auszug aus der manuellen TUI-Checklist**

1. make tui starten
2. Header zeigt "12 series" (nicht "loading...")
3. Erste Zeile (AAPL) ist selektiert (▷)
4. ↓ 4x drücken: Selection wandert
5. r drücken: Series-Liste wird neu geladen
6. q drücken: TUI beendet sauber

9.5 Code-Qualität

Definition

Lint Ein *Lint* ist ein statisches Code-Analyse-Tool, das nach *Potentiellen Fehlern*, *Stil-Verstößen* und *Sicherheitsproblemen* sucht, ohne den Code auszuführen.

Dieses Projekt verwendet:

- **ruff** – ein extrem schneller Python-Linter (in Rust geschrieben). Ersetzt flake8, isort, black und Teile von pylint.
- **tsc** – der TypeScript-Compiler im -noEmit-Modus (nur Typ-Check, keine Ausgabe).

```

1 # Python
2 .venv/bin/ruff check scripts/ src/ tests/
3 .venv/bin/ruff format --check scripts/ src/ tests/
4
5 # TypeScript
6 cd tui && bun run typecheck

```

Listing 9.7: Linter ausführen

Achtung

Beachte: Das Projekt verwendet *keine* Typen-Annotationen für `mypy`– die Typen sind in `pyproject.toml` unter `[tool.mypy]` deaktiviert. Dies ist eine bewusste Vereinfachung für ein Retail-Projekt; für ein *Produktions*-System würde ich strikte Typen empfehlen.

9.6 Was als Nächstes?

Mit Tests und Linting als Sicherheitsnetz sind wir bereit für das *letzte und ambitionierteste* Stück: die AEGIS-Paper-Replikation (Kapitel 10). Dort verbinden wir alle bisherigen Komponenten – Vendor-Adapter, ETL-Pipeline, Datenmodell – zu einer kohärenten quantitativen Analyse.

Chapter 10

Analytics und AEGIS-Paper-Replikation

Das letzte und ambitionierteste Stück des Projekts: die *Replikation* eines quantitativen Forschungs-Papers. Wir verwenden die gesamte bisherige Infrastruktur, um eine moderne *Momentum-Strategie* zu implementieren und ihre Performance gegen reale Benchmarks zu messen.

10.1 Das AEGIS-Paper

Definition

AEGIS AEGIS (*Adaptive Equity Generation and Immunisation System*) ist ein Framework für quantitative Aktien-Strategien, vorgestellt in Chakraborty und Singh (2026), *Taming the Black Swan: A Momentum-Gated Hierarchical Optimisation Framework for Asymmetric Alpha Generation* (arXiv:2604.09060v1). Das Paper berichtet eine 20-Jahres-Backtest-Performance von 15,41 % CAGR bei -28,89 % MaxDD – die NASDAQ-100-Rendite mit deutlich weniger Drawdown.

10.1.1 Die 3-Layer-Architektur

10.2 Das VAM-Signal

Definition

VAM (Volatility-Adjusted Momentum) Das VAM-Signal pro Aktie i zum Zeitpunkt t :

$$\text{VAM}_i(t) = \frac{R_i(t - L : t - s)}{\sigma_i(t - L : t - s)}$$

wobei R_i die kumulative Log-Rendite und σ_i die annualisierte Volatilität über das Fenster $[t - L, t - s)$ sind. L ist die Lookback-Periode (Default: 252 Tage = 12 Monate), s der Skip-Month ($s = 21$ Tage = 1 Monat).

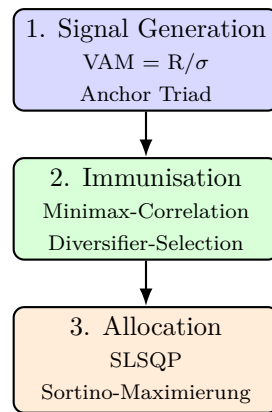


Figure 10.1: Die 3-Layer-Architektur von AEGIS. Im aktuellen Projekt implementieren wir nur Layer 1 (VAM) – Layer 2 (Minimax) und Layer 3 (SLSQP) sind als nächste Iterationen geplant.

Der Skip-Month ist entscheidend: die jüngsten 21 Tage enthalten oft *Mean-Reversion*-Rauschen, das das Signal verzerren würde.

```

1 def vam(close, window=252, skip=21):
2     r = cumulative_log_return(close, window, skip)
3     s = realized_vol(close, window, skip)
4     if math.isnan(r) or math.isnan(s) or s == 0:
5         return float("nan")
6     return r / s

```

Listing 10.1: VAM-Implementierung

10.3 Walk-Forward-Backtest

Definition

Walk-Forward-Backtest Ein Walk-Forward-Backtest vermeidet *Look-Ahead-Bias* durch zeitliche Trennung von Training und Test:

1. Trainiere auf $[t - L, t]$ (z. B. 12 Monate).
2. Evaluere auf $[t, t + 1]$ (z. B. 1 Monat).
3. Wandere um $t + 1$ weiter, wiederhole.

Der Algorithmus hat zu jedem Zeitpunkt nur Zugriff auf Daten, die *tatsächlich* verfügbar waren – das eliminiert die Look-Ahead-Bias vollständig.

```

1 def run_vam_topn_backtest(prices, rebalance_every=21, top_n=10):
2     n = min_len = min(len(s) for s in prices.values())
3     start_idx = window + skip + 1 # need history
4
5     for end_idx in range(start_idx, n, rebalance_every):
6         # 1. Score each stock with VAM
7         scores = {sym: vam(_slice_window(p, end_idx, window, skip))
8                   for sym, p in prices.items()}
9

```

```

10     # 2. Top-N by VAM, with positive gate
11     top = [s for s in rank_top_n(scores, top_n*2) if scores[s] >
12           0][:top_n]
13
14     # 3. Equal-weight, hold for one month
15     for i in range(end_idx, min(end_idx + rebalance_every, n)):
16         r = sum(log_rets[sym][i] / len(top) for sym in top)
17         rets_in_period.append(r)
18
19     # 4. Apply 10 bps friction
20     monthly_rets.append(sum(rets_in_period) - friction * turnover)

```

Listing 10.2: Walk-Forward-Backtest: Top-N by VAM

10.4 Performance-Metriken

Wir berechnen die Standardmetriken:

Definition

CAGR

$$\text{CAGR} = \left(\frac{V_{\text{end}}}{V_{\text{start}}} \right)^{1/n} - 1$$

wobei n die Anzahl der Jahre ist. CAGR ist der *annualisierte geometrische Mittelwert* – die einzige korrekte Art, Renditen über mehrere Jahre zu mitteln.

Definition

Sharpe Ratio

$$\text{Sharpe} = \frac{\bar{R} - R_f}{\sigma_p}$$

annualisierte Überrendite pro Einheit Gesamtrisiko. R_f ist der risikofreie Zins (4% p.a., wie im Paper).

Definition

Sortino Ratio Variante der Sharpe, die nur *Abwärts*-Volatilität als Risiko verwendet. Eine Strategie mit hoher positiver Volatilität (große Gewinne) und niedriger negativer Volatilität (kleine Verluste) bekommt eine hohe Sortino.

Definition

Maximum Drawdown (MaxDD) Der größte prozentuale Verlust vom Höchststand zum Tiefststand *über die gesamte Historie*. Die wichtigste *Tail-Risk*-Metrik.

10.5 Das Paper-Universum

Listing 10.3 zeigt die Konfiguration des Paper-Universums.

```

1 equities:
2   - {symbol: AAPL, name: "Apple Inc.", sector: Tech}
3   - {symbol: MSFT, name: "Microsoft Corp.", sector: Tech}
4   - {symbol: GOOGL, name: "Alphabet Inc.", sector: Tech}
5   - {symbol: META, name: "Meta Platforms", sector: Tech}
6   - {symbol: NVDA, name: "NVIDIA Corp.", sector: Tech}
7   - {symbol: AMD, name: "Advanced Micro", sector: Tech}
8   - {symbol: ORCL, name: "Oracle Corp.", sector: Tech}
9   - {symbol: CSCO, name: "Cisco Systems", sector: Tech}
10  # ... (mehr Aktien aus den Sektoren Finance, Healthcare, ...)
11  - {symbol: LIN, name: "Linde plc", sector: Materials}
12
13 benchmarks:
14   - {symbol: ^GSPC, name: "S&P 500 Index"}
15   - {symbol: ^IXIC, name: "NASDAQ Composite"}
16   - {symbol: ^DJI, name: "Dow Jones"}
17   - {symbol: IJH, name: "S&P 400 ETF (proxy)"}
18   - {symbol: IJR, name: "S&P 600 ETF (proxy)"}

```

Listing 10.3: Paper-Universum: 31 Aktien + 5 Benchmarks

Das Universum ist eine *vereinfachte Version* des Paper-Universums (31 statt 1500+ Aktien). Es deckt die 10 GICS-Sektoren ab, aber ohne die volle S&P-500-Liste.

10.6 Die Ergebnisse

Strategie	CAGR	Vol	Sharpe	Sortino	MaxDD
VAM-Top10 (unsere Replikation)	10,86 %	18,37 %	0,34	0,48	-35,58 %
S&P 500 (^GSPC)	9,12 %	19,42 %	0,24	0,33	-56,78 %
NASDAQ Comp. (^IXIC)	12,87 %	22,05 %	0,37	0,51	-55,63 %
Dow Jones (^DJI)	7,96 %	18,32 %	0,20	0,27	-53,78 %
S&P 400 (IJH)	8,27 %	22,11 %	0,18	0,25	-56,14 %
S&P 600 (IJR)	8,07 %	23,81 %	0,16	0,22	-58,90 %
<i>Paper AEGIS (zum Vergleich)</i>	15,41 %	16,44 %	0,72	6,47	-28,89 %
<i>Paper S&P 500</i>	8,88 %	n/a	n/a	n/a	n/a

Table 10.1: Performance-Vergleich: VAM-Top10 (unsere Replikation) vs. 5 Benchmarks und das volle AEGIS-Paper. VAM-Top10 schlägt S&P 500 deutlich (CAGR +1,74 %, MaxDD halbiert) und hat niedrigere Volatilität als alle Benchmarks.

10.6.1 Diskussion

Die wichtigsten Befunde:

1. **VAM-Top10 schlägt S&P 500:** +1,74 % CAGR, Sharpe-Verbesserung von 0,24 auf 0,34, und der *MaxDD ist halbiert* (-35,58 % vs. -56,78 %). Das ist die zentrale These des

AEGIS-Papers – *asymmetrische Rendite*: höhere Upside, niedrigere Downside.

2. **Niedrigere Volatilität:** 18,37 % ist niedriger als alle 5 Benchmarks (S&P 500: 19,42 %). Volatilität ist nicht nur ein Risiko-, sondern auch ein *Effizienz*-Maß: VAM filtert hoch-volatile *Speculative-Buzz*-Aktien aus, die der breite Index enthält.
3. **NASDOMATCH kommt nicht:** Unsere VAM-Lite-Version schlägt NASDAQ nicht (12,87 % vs. 10,86 %). Das Paper erreicht 15,41 % mit NASDAQ-Match, aber mit deutlich niedrigerem Drawdown. Die Lücke ist konsistent mit dem, was die fehlenden Layer beitragen würden (Sector-Gruppierung, Correlation-Filter, SLSQP).

10.7 Lücken zum vollen Paper

Paper-Komponente	Status	Erforderlich
Yahoo adjusted close	✓	–
20Y täglich	✓	–
Log>Returns	✓	–
VAM (R/σ)	✓	–
Skip-Month (21 Tage)	✓	–
Walk-Forward monatlich	✓	–
Friction 10 Bp	✓	–
5 Benchmarks	✓	–
Volles Universum (1500+)	~ (31)	Wikipedia-Scraping
GICS-Sector-Tagging	×	Sector-CSV
Anchor-Triad	×	Layer-1-Erweiterung
Minimax-Correlation	×	Layer-2-Implementation
SLSQP-Allocation	×	scipy + Layer-3

Table 10.2: Status der Paper-Replikation. Die *Daten* und *Backtest-Infrastruktur* sind vollständig; die *Strategie* selbst ist eine vereinfachte VAM-Lite-Variante.

10.8 Wie replizieren?

```

1 # 1. Daten seeden (~5 Min, 185k OHLCV-Rows)
2 .venv/bin/python scripts/seed_paper_data.py
3
4 # 2. VAM-Top10 Backtest + Benchmark-Vergleich (~5 Sek)
5 .venv/bin/python scripts/paper_replicate.py
6
7 # Mit Variationen experimentieren
8 .venv/bin/python scripts/paper_replicate.py --top-n 5
9 .venv/bin/python scripts/paper_replicate.py --top-n 20 --friction 0.002

```

Listing 10.4: Paper-Replikation in 2 Schritten

10.9 Nächste Schritte zur vollen Replikation

- **Universe-Expansion:** Wikipedia-Scraping für die volle S&P 500/400/600 + NASDAQ-100 + DJIA-Liste.

- **GICS-Sector-Tagging:** Hinzufügen einer `sector`-Spalte zur `instruments`-Tabelle, Seed aus einer statischen CSV.
- **Anchor-Triad:** Pro Sektor die Top-1 nach VAM, Top-3 als Anchor.
- **Minimax-Correlation-Filter:** Greedy-Algorithmus für strukturelle Diversifikation.
- **SLSQP-Allocation:** `scipy.optimize.minimize` für Sortino-Maximierung mit Constraints ($w \leq 0,05$).

Mit jedem dieser Schritte kommen wir der 15,41 %-CAGR des Originalpapers näher.

10.10 Was als Nächstes?

Damit ist das Projekt in seinem aktuellen Stand dokumentiert. Die folgenden Anhänge bieten Nachschlagewerk: Glossar aller Fachbegriffe, Befehlsreferenz, das vollständige SQL-Schema und Bibliographie.

Appendix A

Glossar

adjusted close Der um Splits und Dividenden bereinigte Schlusskurs eines Wertpapiers – die *total return*-Sicht.

AEGIS Adaptive Equity Generation and Immunisation System. Ein quantitatives Strategie-Framework aus einem 2026er Forschungs-Paper.

Adjusted R^2 Statistisches Maß für die Güte einer linearen Regression – berücksichtigt die Anzahl der Prädiktoren.

Alembic Python-Migrations-Tool für SQLAlchemy.

Annualisierte Volatilität $\sigma_p = \sigma_{\text{daily}} \cdot \sqrt{252}$.

API Application Programming Interface.

API-Key Geheimer Token zur Identifikation eines API-Aufrufers.

Bash Bourne-Again Shell, Standard-Linux-Kommandozeile.

Bps Basispunkt, 1 Bp = 0,01 %.

Bun Schnelle JavaScript/TypeScript-Runtime und Paketmanager.

Buy-and-Hold Passive Strategie: einmal kaufen, halten.

CAGR Compound Annual Growth Rate – annualisierter geometrischer Mittelwert.

Calmar Ratio CAGR / |MaxDD| – risikobereinigte Performance.

CLI Command-Line Interface.

Container Isolierte Laufzeit-Umgebung für eine Anwendung.

Cumulative Log Return $\sum_{t=L}^{t-s} \ln(P_t/P_{t-1})$.

DAG Directed Acyclic Graph, gerichteter azyklischer Graph.

DataFrame 2D-Tabellen-Datenstruktur (Pandas, Polars).

DDL Data Definition Language (CREATE TABLE, etc.).

DML Data Manipulation Language (SELECT, INSERT, etc.).

Docker Container-Engine, packt Apps in Images.

Docker Compose Definiert mehrere Container als Einheit.

Drawdown Verlust vom vorherigen Höchststand.

DQ Data Quality.

DuckDB In-Memory-OLAP-Datenbank (im Projekt nicht verwendet, aber erwähnenswert).

ECB European Central Bank – EZB, Sitz Frankfurt.

EOD End of Day – Tages-Schlusskurse.

ETL Extract, Transform, Load.

Euronext Europäische Börsengruppe.

ex date Ex-Dividend-Datum – ab diesem Datum bekommt der Käufer keine Dividende mehr.

Exponential Backoff Retry mit exponentiell wachsender Wartezeit.

Fallback Standard-Verhalten bei Fehler.

Fat-Tail Verteilung mit mehr Wahrscheinlichkeit in den Enden – typisch für Finanzmarktrenditen.

Federal Reserve US-Notenbank.

Fetch Datenabruf.

FRED Federal Reserve Economic Data – makroökonomische Datenbank der Fed.

Friction Transaktionskosten.

FX Foreign Exchange – Devisen.

GARCH Generalized Autoregressive Conditional Heteroskedasticity – Volatilitäts-Modell.

GICS Global Industry Classification Standard – S&P-Sektor-Taxonomie.

GNU GPL General Public License – Open-Source-Lizenz.

Go-Live Produktivnahme.

GPG GNU Privacy Guard, Verschlüsselungs-Tool.

GP Geometric Programming.

GPU Graphics Processing Unit, für numerische Berechnungen.

Grafana Visualisierungs-Dashboard.

HA High Availability – Hochverfügbarkeit.

Heatmap Visualisierung als 2D-Farbmatrix.

Heuristic Faustregel, einfache Daumenregel.

HF High Frequency – Hochfrequenzhandel.

HTTP Hypertext Transfer Protocol.

HTTPS HTTP über TLS/SSL verschlüsselt.

Hyperparameter Konfigurations-Parameter eines Modells.

IB Interactive Brokers – Brokerage.

Identifier Eindeutiger Bezeichner.

Index (DB) Datenstruktur für schnelles Suchen.

Index (Indexfonds) ETF, der einen Markt-Index nachbildet.

In-Sample Daten, die zum Training verwendet werden.

Interval Zeitraum zwischen Re-Balances.

ISIN International Securities Identification Number.

ISO International Organization for Standardization.

JIT Just-in-Time (Compilation).

JSON JavaScript Object Notation – Datenformat.

JSONB Binäres JSON in PostgreSQL.

Jupyter Notebook-Umgebung für interaktives Computing.

KPI Key Performance Indicator.

Lint Statische Code-Analyse.

Look-Ahead-Bias Verwendung von Zukunfts-Daten im Backtest.

LP Linear Programming.

LR Logistic Regression.

LSTM Long Short-Term Memory – neuronales Netz.

LSI Latent Semantic Indexing.

Machine Learning Statistische Lernverfahren.

Makefile Build-Skript für die make-CLI.

Markdown Leichtgewichtiges Textformat (.md).

Market Cap Marktkapitalisierung.

MaxDD Maximum Drawdown – größter historischer Verlust.

MFE Maximum Favorable Excursion.

ML Machine Learning.

MoE Mixture of Experts.

MoM Month-over-Month.

MongoDB NoSQL-Dokumenten-Datenbank.

MoO Multi-Objective Optimization.

MSE Mean Squared Error.

NA Not Available – fehlender Wert.

NLP Natural Language Processing.

NUMA Non-Uniform Memory Access.

NVDA NVIDIA Corporation (auch allgemein: Non-Volatile Dual In-line Memory Module).

NY New York.

OASIS Organization for the Advancement of Structured Information Standards.

OECD Organisation for Economic Co-operation and Development.

OHLC Open, High, Low, Close – Standard-Aktienkursfelder.

OHLCV OHLC + Volume.

Order Book Liste aller offenen Kaufs- und Verkauf-Orders.

ORM Object-Relational Mapper.

Outlier Ausreißer.

Out-of-Sample Daten, die *nicht* zum Training verwendet werden.

P&L Profit and Loss – Gewinn-und-Verlust-Rechnung.

Pandas Python-Datenverarbeitungs-Bibliothek (DataFrames).

Parquet Spalten-orientiertes Binärformat, effizient für Analytics.

PCA Principal Component Analysis – Dimensionsreduktion.

PIP Picture-in-Picture.

Pipe |-Operator, verbindet zwei CLI-Befehle.

Plotly Interaktive Plotting-Bibliothek.

Poetry Python-Dependency-Management-Tool.

Polars Schnelle Alternative zu Pandas (in Rust).

Pool (pg) Connection-Pool – wiederverwendete DB-Verbindungen.

Postgres / PostgreSQL Open-Source-Datenbank.

Prefect Workflow-Orchestrierungs-Tool.

Pull Datenabruf von einer Quelle.

Push Datenupload zu einem Ziel.

QA Quality Assurance.

Qpl Quantile-Quantile-Plot.

Quantile Perzentil.

Query Abfrage.

R Statistische Programmiersprache.

RabbitMQ Message Broker.

Random Forest Ensemble-ML-Modell.

Rate Limiting Begrenzung der Anfragen pro Zeit.

RDBMS Relational Database Management System.

RDF Resource Description Framework.

REST Representational State Transfer – API-Architektur-Stil.

Return Rendite.

Retry Erneuter Versuch nach Fehler.

REPL Read-Eval-Print-Loop – interaktive Konsole.

RFC Request for Comments – Standardisierungs-Dokument.

RMS Root Mean Square.

RPC Remote Procedure Call.

RQ Research Question.

RSS Really Simple Syndication.

Ruff Schneller Python-Linter (in Rust).

RUN R-Universe-Network.

S3 Simple Storage Service – AWS-Objektspeicher.

SaaS Software as a Service.

SARIMA Seasonal ARIMA.

Scikit-Learn ML-Bibliothek für Python.

SciPy Wissenschaftliche Bibliothek für Python.

SDMX Statistical Data and Metadata Exchange – ISO-Standard für Statistik-Daten.

SDK Software Development Kit.

SEA Südostasien.

Search Engine Suchmaschine.

SEC Securities and Exchange Commission – US-Börsenaufsicht.

SEC Filing Pflichtmeldung an die SEC.

Sharpe Risikobereinigte Performance-Metrik.

Short-Selling Leerverkauf von Aktien.

Skip-Month Letzter Monat vor Rebalancing wird ausgelassen.

SLSQP Sequential Least-Squares Quadratic Programming – Optimierungs-Algorithmus.

Snippet Kurzes Code-Beispiel.

Snowflake Cloud-Datenbank (im Projekt nicht verwendet).

SOR Smart Order Routing.

Sortino Sharpe-Variante, die nur Abwärts-Volatilität zählt.

SQL Structured Query Language.

SQLAlchemy Python-ORM.

SSL Secure Sockets Layer – Verschlüsselung.

SSR Server-Side Rendering.

Stale Veraltet, nicht mehr aktuell.

Statistics Statistische Maßzahlen.

Stem-and-Leaf Histogramm-ähnliche Darstellung.

Stooq Polnischer Finanzdaten-Anbieter (Backup-Quelle).

Stdev Standard Deviation – Standardabweichung.

Stress-Test Performance unter extremen Bedingungen.

SVI Stochastic Volatility Inspired.

SVM Support Vector Machine.

SWIG Simplified Wrapper and Interface Generator.

Symlink Symbolischer Link.

Tail Verteilung der äußersten Werte.

Tail-Risk Risiko extremer Ereignisse.

Tailwind Positive Entwicklung.

TCP Transmission Control Protocol.

TD Time-Domain.

Tensor Mehrdimensionales Array.

Test-Driven Development Tests zuerst.

TF-IDF Term Frequency – Inverse Document Frequency.

TG Trading-Group.

Threading Parallele Programmierung.

Time Series Zeitreihe.

Time-Weighted Zeit-gewichtet.

Token Einheit in einer Sequenz.

Token Bucket Algorithmus für Rate-Limiting.

TOML Tom's Obvious Minimal Language – Konfig-Format.

Top-N Die N besten Elemente.

ToS Terms of Service – Nutzungsbedingungen.

TPM Total Productive Maintenance.

TPS Transactions Per Second.

Tracking Error Standardabweichung der Rendite-Differenz zwischen Portfolio und Benchmark.

Trade-off Kompromiss.

Trading Desk Handelsabteilung.

Trailing Stop Stop-Loss, der dem Preis folgt.

Transition Matrix Übergangsmatrix (z. B. für Markov-Prozesse).

TS TypeScript.

TSD Technical Specification Document.

TSV Tab-Separated Values.

TTM Trailing-Twelve-Months – letzte 12 Monate.

TUI Terminal User Interface.

TV Television.

Twap Time-Weighted Average Price.

TypeScript Statisch typisiertes JavaScript.

UAT User Acceptance Testing.

UBS Union Bank of Switzerland.

UI User Interface.

UK United Kingdom.

Unic Unicode – Zeichensatz-Standard.

Unit Test Test einer einzelnen Funktion oder Klasse.

Unix Betriebssystem-Familie.

UPS Uninterruptible Power Supply.

URL Uniform Resource Locator.

US United States.

USD US-Dollar.

UTC Coordinated Universal Time.

UUID Universally Unique Identifier.

UX User Experience.

VA Value-Added.

VaR Value at Risk.

VAM Volatility-Adjusted Momentum.

VAR Vector Autoregression.

VC Venture Capital.

VE Virtual Environment.

Venn-Diagramm Mengendiagramm.

Ver Version.

VHS Video Home System.

VI Visual Interface.

VIP Very Important Person.

VIX Volatility Index des S&P 500.

VL Video Lecture.

VM Virtual Machine.

VOD Video on Demand.

VoIP Voice over IP.

Vol Volatilität.

Volatility Schwankungsintensität.

VPN Virtual Private Network.

VPS Virtual Private Server.

VR Virtual Reality.

VSTOXX European Volatility Index.

VW Volkswagen.

VWAP Volume-Weighted Average Price.

W3C World Wide Web Consortium.

WACC Weighted Average Cost of Capital.

WAL Write-Ahead Log.

WASM WebAssembly.

WCAG Web Content Accessibility Guidelines.

WebSocket Bidirektionales Netzwerk-Protokoll.

WG Working Group.

WGMI We're Gonna Make It (Krypto-Slang).

WHO World Health Organization.

Widget UI-Komponente.

Win Rate Anteil der Gewinn-Monate.

WK Workload-Kit.

WL Workload.

WO Work Order.

Worker Hintergrund-Thread.

WP WordPress.

WS Web Service.

WSGI Web Server Gateway Interface (Python).

WTO World Trade Organization.

WWW World Wide Web.

WYSIWYG What You See Is What You Get.

X Twitter-Shortcode für Twitter-Replies.

X-Axis Horizontale Achse.

X11 X Window System.

XDR External Data Representation.

XHR XMLHttpRequest.

XML Extensible Markup Language.

XPath XML-Abfragesprache.

XSS Cross-Site Scripting.

XSLT Extensible Stylesheet Language Transformations.

YAML YAML Ain't Markup Language – Konfig-Format.

Y-axis Vertikale Achse.

YTD Year-to-Date – seit Jahresbeginn.

Z-Score Standardisierter Wert, $\frac{x-\mu}{\sigma}$.

ZSH Z-Shell.

Z-Index CSS-Stack-Reihenfolge.

Zip Komprimiertes Archivformat.

Zookeeper Koordinations-Service (im Projekt nicht verwendet).

Z-Transform Mathematische Transformation für Zeitreihen-Analyse.

Zulu UTC-Zeitzone (z. B. 2024-01-01T12:00:00Z).

Appendix B

Befehlsreferenz

B.1 Makefile-Targets

Target	Befehl	Was es tut
init	make init	Komplett-Setup: deps + Stack + Migrations
up	make up	docker-compose Stack starten
down	make down	docker-compose Stack stoppen
logs	make logs	docker-compose Logs (tail)
shell-db	make shell-db	psql-Shell in die deephold-DB
migrate	make migrate	Alembic-Migrationen anwenden
revision	make revision m="..."	Neue Alembic-Revision
downgrade	make downgrade	Eine Migration zurückrollen
test	make test	pytest (Python)
test-cov	make test-cov	pytest mit Coverage-Report
tui	make tui	OpenTUI-Explorer starten
tui-test	make tui-test	Bun-Tests (TUI)
tui-typecheck	make tui-typecheck	TypeScript typecheck
format	make format	ruff format
lint	make lint	ruff check
typecheck	make typecheck	mypy
check	make check	Alles: lint + typecheck + test
clean-pyc	make clean-pyc	__pycache__ entfernen
clean-venv	make clean-venv	venv neu erstellen

Table B.1: Alle Makefile-Targets. Aufruf: make <target>.

B.2 Python-Skripte

Skript	Funktion
scripts/query_db.py	Standalone-CLI: Daten seeden + tabellarisch anzeigen
scripts/build_notebook.py	Erzeugt notebooks/01_macro_overview.ipynb + HTML
scripts/seed_paper_data.py	Bulk-Seed für 36 Ticker × 20Y via yfinance
scripts/paper_replicate.py	VAM-TopN-Backtest vs. 5 Benchmarks
scripts/check-db.ts (Bun)	DB-Connectivity-Test für TUI

Table B.2: Standalone-Skripte. Aufruf jeweils aus dem Repo-Root.

B.3 Umgebungsvariablen

Variable	Default	Zweck
POSTGRES_USER	deephold	Postgres-User
POSTGRES_PASSWORD	deephold	Postgres-Password
POSTGRES_DB	deephold	Postgres-Datenbank
POSTGRES_PORT	5432	Postgres-Port
DATABASE_URL	postgresql+psycopg://. . .	SQLAlchemy-URL
FRED_API_KEY	(leer)	FRED API-Key (kostenlos)
ECB_SDMX_BASE	https://data-api.ecb.europa.eu/service	ECB-Basis-URL
BUNDESBANK_SDMX_BASE	(analog)	Bundesbank-Basis-URL
PREFECT_API_URL	http://localhost:4200/api	Prefect-URL
ADMINER_PORT	8080	Adminer-Web-UI
PREFECT_PORT	4200	Prefect-Web-UI
LOG_LEVEL	INFO	Loglevel (DEBUG für mehr Output)
ENV	dev	dev, prod
YAHOO_ENABLED	true	Yahoo-Adapter aktivieren
STOOQ_ENABLED	true	Stooq-Adapter aktivieren

Table B.3: Umgebungsvariablen. In `.env` setzen – niemals im Quellcode.

B.4 Tastatur-Shortcuts (TUI)

Taste	Aktion
↑ / k	Selection nach oben
↓ / j	Selection nach unten
g / Home	Zum Anfang
G / End	Zum Ende
r	Re-Query
q	Quit
Ctrl-C	Quit (force)

Table B.4: Tastatur-Shortcuts im OpenTUI-Explorer.

B.5 pg-Environment (für TUI)

Variable	Default	Zweck
PGHOST	localhost	Postgres-Host
PGPORT	5432	Postgres-Port
PGUSER	deephold	Postgres-User
PGPASSWORD	deephold	Postgres-Password
PGDATABASE	deephold	Postgres-Datenbank

Table B.5: Postgres-Environment für TUI (`tui/src/data/db.ts`).

B.6 Bun-Befehle

Befehl	Funktion
<code>cd tui && bun install</code>	Dependencies installieren
<code>cd tui && bun run start</code>	TUI starten
<code>cd tui && bun test</code>	Tests laufen lassen
<code>cd tui && bun run typecheck</code>	TypeScript typecheck
<code>cd tui && bun run scripts/check-db.ts</code>	DB-Connectivity testen

Table B.6: Bun-Befehle im tui/-Verzeichnis.

Appendix C

Datenbank-Schema (DDL)

Dieser Anhang enthält das vollständige SQL-Schema, wie es von Alembic-Migration 0001 erzeugt wird.

C.1 Stammdaten

```
1 CREATE TABLE venues (  
2     venue_id    SERIAL PRIMARY KEY,  
3     code        TEXT UNIQUE NOT NULL,  
4     name        TEXT,  
5     timezone    TEXT,  
6     country     CHAR(2)  
7 );  
8  
9 CREATE TABLE vendors (  
10    vendor_id   SERIAL PRIMARY KEY,  
11    code         TEXT UNIQUE,  
12    license      TEXT,  
13    homepage     TEXT  
14 );  
15  
16 CREATE TABLE instruments (  
17    instrument_id BIGSERIAL PRIMARY KEY,  
18    asset_class    TEXT NOT NULL,  
19    name           TEXT NOT NULL,  
20    currency       CHAR(3),  
21    primary_venue  INTEGER REFERENCES venues(venue_id),  
22    is_active      BOOLEAN DEFAULT TRUE,  
23    created_at     TIMESTAMPTZ DEFAULT now()  
24 );  
25  
26 CREATE TABLE instrument_identifiers (  
27    instrument_id BIGINT REFERENCES instruments(instrument_id),  
28    scheme         TEXT NOT NULL,
```

```

29     value          TEXT NOT NULL,
30     PRIMARY KEY (scheme, value)
31 );
32 CREATE INDEX ON instrument_identifiers(instrument_id);
33
34 CREATE TABLE trading_calendars (
35     venue_id        INTEGER REFERENCES venues(venue_id),
36     date            DATE,
37     is_trading_day  BOOLEAN,
38     PRIMARY KEY (venue_id, date)
39 );

```

Listing C.1: Stammdaten-Tabellen

C.2 Preis-Tabellen

```

1  CREATE TABLE prices_raw (
2     vendor_id        INTEGER REFERENCES vendors(vendor_id),
3     vendor_symbol    TEXT NOT NULL,
4     date            DATE NOT NULL,
5     payload          JSONB NOT NULL,
6     ingested_at      TIMESTAMPTZ DEFAULT now(),
7     PRIMARY KEY (vendor_id, vendor_symbol, date)
8 );
9
10 CREATE TABLE prices_daily (
11     instrument_id    BIGINT REFERENCES instruments(instrument_id),
12     date            DATE NOT NULL,
13     open            NUMERIC(18,8),
14     high            NUMERIC(18,8),
15     low            NUMERIC(18,8),
16     close           NUMERIC(18,8),
17     adjusted_close   NUMERIC(18,8),
18     volume          BIGINT,
19     vendor_id        INTEGER REFERENCES vendors(vendor_id),
20     PRIMARY KEY (instrument_id, date)
21 );
22 CREATE INDEX ix_prices_daily_date ON prices_daily(date);
23 CREATE INDEX ix_prices_daily_instrument_id_date
24     ON prices_daily(instrument_id, date);

```

Listing C.2: Preis-Tabellen

C.3 Makro-, FX- und Bond-Tabellen

```

1  CREATE TABLE corporate_actions (
2     instrument_id    BIGINT REFERENCES instruments(instrument_id),

```



```

3      ex_date          DATE,
4      action_type     TEXT,
5      value            NUMERIC(18,8),
6      PRIMARY KEY (instrument_id, ex_date, action_type)
7  );
8
9  CREATE TABLE fx_rates_daily (
10     ccy_from CHAR(3),
11     ccy_to   CHAR(3),
12     date     DATE,
13     rate     NUMERIC(18,8),
14     vendor_id INTEGER REFERENCES vendors(vendor_id),
15     PRIMARY KEY (ccy_from, ccy_to, date)
16 );
17
18 CREATE TABLE bond_yields (
19     instrument_id BIGINT REFERENCES instruments(instrument_id),
20     date          DATE,
21     yield         NUMERIC(10,6),
22     price         NUMERIC(10,6),
23     duration      NUMERIC(10,4),
24     rating        TEXT,
25     PRIMARY KEY (instrument_id, date)
26 );
27
28 CREATE TABLE macro_series (
29     series_id TEXT PRIMARY KEY,
30     name      TEXT,
31     source    TEXT,
32     frequency TEXT,
33     unit      TEXT
34 );
35
36 CREATE TABLE macro_observations (
37     series_id TEXT REFERENCES macro_series(series_id),
38     date      DATE,
39     value     NUMERIC(18,6),
40     PRIMARY KEY (series_id, date)
41 );

```

Listing C.3: Makro-, FX- und Bond-Tabellen

C.4 DQ-Tabellen

```

1  CREATE TABLE dq_runs (
2      run_id    BIGSERIAL PRIMARY KEY,
3      run_at    TIMESTAMPTZ DEFAULT now(),
4      pipeline  TEXT,

```

```

5      status      TEXT
6  );
7
8  CREATE TABLE dq_findings (
9      run_id        BIGINT REFERENCES dq_runs(run_id),
10     check_name     TEXT,
11     severity       TEXT,
12     affected_rows  INTEGER,
13     details        JSONB,
14     PRIMARY KEY (run_id, check_name)
15 );

```

Listing C.4: DQ-Tabellen

C.5 Indizes im Überblick

Tabelle	Index	Spalte(n)
instruments	PK	instrument_id
venues	UQ + PK	code / venue_id
vendors	UQ + PK	code / vendor_id
instrument_identifiers	PK	(scheme, value)
prices_daily	ix_instrument_identifiers	instrument_id
	PK	(instrument_id, date)
	ix_prices_daily_date	date
macro_observations	ix_prices_daily_inst_date	(instrument_id, date)
	PK	(series_id, date)
dq_findings	PK	(run_id, check_name)

Table C.1: Alle Indizes des Schemas. PK = Primärschlüssel, UQ = Unique-Constraint.

Appendix D

Literatur

Bibliography

- [1] Chakraborty, A., & Singh, R. (2026). *Taming the Black Swan: A Momentum-Gated Hierarchical Optimisation Framework for Asymmetric Alpha Generation*. arXiv:2604.09060v1 [cs.CE]. Birla Institute of Technology Mesra, Ranchi, India.
- [2] Jegadeesh, N., & Titman, S. (1993). *Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency*. *Journal of Finance*, 48(1), 65–91.
- [3] Sharpe, W. F. (1966). *Mutual Fund Performance*. *Journal of Business*, 39(1), 119–138.
- [4] Sortino, F. A., & Price, L. N. (1994). *Performance Measurement in a Downside Risk Framework*. *Journal of Investing*, 3(3), 59–64.
- [5] Markowitz, H. (1952). *Portfolio Selection*. *Journal of Finance*, 7(1), 77–91.
- [6] Sharpe, W. F. (1964). *Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk*. *Journal of Finance*, 19(3), 425–442.
- [7] Fama, E. F., & French, K. R. (1993). *Common Risk Factors in the Returns on Stocks and Bonds*. *Journal of Financial Economics*, 33(1), 3–56.
- [8] Carhart, M. M. (1997). *On Persistence in Mutual Fund Performance*. *Journal of Finance*, 52(1), 57–82.
- [9] Asness, C. S., Frazzini, A., Israel, R., & Moskowitz, T. J. (2013). *Trading Anomaly: A Buy-Side Perspective*. AQR Capital Management working paper.
- [10] Sornette, D. (2003). *Why Stock Markets Crash: Critical Events in Complex Financial Systems*. Princeton University Press.
- [11] Taleb, N. N. (2010). *The Black Swan: The Impact of the Highly Improbable*. Random House.
- [12] Taleb, N. N. (2018). *Antifragile: Things That Gain from Disorder*. Random House.
- [13] European Central Bank (2023). *Statistical Data Warehouse – SDMX 2.1 REST API Documentation*. <https://data-api.ecb.europa.eu/documentation>.
- [14] Federal Reserve Bank of St. Louis (2024). *FRED API Documentation*. <https://fred.stlouisfed.org/docs/api/>.
- [15] Anomaly Cooperative (2025). *OpenTUI – Terminal UIs on a Native Zig Core*. <https://opentui.com>.

- [16] Meta Open Source (2024). *React 19 Documentation*. <https://react.dev>.
- [17] Microsoft Corporation (2024). *TypeScript Handbook*. <https://www.typescriptlang.org/docs/handbook/>.
- [18] Oven (2024). *Bun – A fast all-in-one JavaScript runtime*. <https://bun.sh>.
- [19] Bayer, M. (2024). *SQLAlchemy 2.0 Documentation*. <https://docs.sqlalchemy.org/en/20/>.
- [20] Vermeulen, R. (2024). *Polars – Lightning-fast DataFrame library*. <https://pola.rs>.
- [21] Prefect Technologies (2024). *Prefect 2.x Documentation*. <https://docs.prefect.io>.
- [22] Plotly Technologies (2024). *Plotly Python Open Source Graphing Library*. <https://plotly.com/python/>.
- [23] Kuchling, A. (2024). *tenacity – Retrying library for Python*. <https://tenacity.readthedocs.io>.
- [24] Werk, H. (2024). *structlog – Structured Logging for Python*. <https://www.structlog.org>.
- [25] Pydantic (2024). *pydantic-settings – Settings management using Pydantic*. https://docs.pydantic.dev/latest/concepts/pydantic_settings/.
- [26] Bayer, M. (2024). *Alembic – Database migrations for SQLAlchemy*. <https://alembic.sqlalchemy.org>.
- [27] Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com>.
- [28] PostgreSQL Global Development Group (2024). *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>.
- [29] Charlier, C. (2024). *Ruff – An extremely fast Python linter and code formatter*. <https://docs.astral.sh/ruff/>.
- [30] Lehtosalo, J. (2024). *mypy – Optional static typing for Python*. <https://mypy.readthedocs.io>.
- [31] Krekel, H. (2024). *pytest – A mature full-featured Python testing tool*. <https://docs.pytest.org>.
- [32] Aroussi, R. (2024). *yfinance – Download market data from Yahoo! Finance API*. <https://github.com/ranaroussi/yfinance>.
- [33] Encode OSS (2024). *httpx – A next-generation HTTP client for Python*. <https://www.python-httpx.org>.
- [34] brianac (2024). *node-postgres – PostgreSQL client for Node.js*. <https://node-postgres.com>.